

LTM_JAVA_MCQ_PRACTICE_FOR_TRAINER

Test Summary

- No. of Sections: 1
- No. of Questions: 446
- Total Duration: 9999 min

Section 1 - LTM_JAVA_MCQ_PRACTICE_FOR_TRAINER

Section Summary

- No. of Questions: 446
- Duration: 9999 min

Additional Instructions:

None

Q1. In Spring Boot JPA, what does the @PersistenceContext annotation represent?

It injects a reference to the EntityManager into a Spring bean.

It defines the persistence unit for the JPA entity manager.

It specifies the database schema for entity mappings.

It configures the transactional behavior of JPA repositories.

Q2. What is the purpose of the @ExceptionHandler annotation in Spring MVC?

It handles exceptions thrown by controller methods

It defines a custom exception class

It automatically enables global exception handling

It configures application logging

Q3. What is the primary purpose of using DTOs (Data Transfer Objects) in a Spring Boot application with a multi-tiered architecture?

DTOs facilitate direct communication between the presentation layer and the database, reducing the need for intermediary services.

DTOs encapsulate domain logic and business rules, ensuring data consistency and integrity across the application.

DTOs improve performance by reducing the size of data transferred between client and server, minimizing network overhead.

DTOs serve as lightweight data containers for transferring data between different layers of the application, promoting separation of concerns and decoupling.

Q4. When debugging a Spring Boot application, what does a "502 Bad Gateway" error typically indicate?

Server overload

Database connection failure

Proxy or load balancer issue

Memory leak

Q5. What issue can occur if the mappedBy attribute is incorrectly configured in a bidirectional one-to-many relationship?

Hibernate may generate an incorrect database schema or create duplicate relationship mappings

The application automatically switches to lazy loading

JSON serialization always causes infinite recursion

Foreign keys are automatically removed from the database

Q6. What is the primary purpose of the 'cascade' attribute in a JPA association mapping?

It specifies the fetch strategy for associated entities during querying.

It determines the propagation behavior of operations (e.g., save, delete) from parent to child entities.

It configures the naming strategy for foreign key columns in the database schema.

It indicates the type of join used to establish the association between entities.

Q7. Which of the following configurations correctly sets up a one-to-many relationship where Order is the parent and OrderItem is the child entity in Spring Boot JPA?

@OneToMany(mappedBy = "order", cascade = CascadeType.ALL) in Order and @ManyToOne in OrderItem.

@OneToMany in Order and @ManyToOne(mappedBy = "order") in OrderItem.

@OneToMany(cascade = CascadeType.ALL) in Order and @ManyToOne in OrderItem.

@OneToMany in Order and @ManyToOne(cascade = CascadeType.ALL) in OrderItem.

Q8. What is the purpose of the @Transactional annotation in Spring?

It defines a transactional boundary for a method or class

It marks an entity as transaction-safe

It creates database tables automatically

It enables dependency injection

Q9. What is the primary reason to use @OneToMany(mappedBy = "parent") in the parent entity of a one-to-many association?

To indicate that the parent entity is responsible for managing the foreign key.

To specify that the child entity owns the relationship.

To avoid creating a join table for the association.

To ensure that the association is lazily loaded.

Q10. In a bidirectional One-to-One JPA relationship, what does the mappedBy attribute indicate?

The name of the relationship field in the owning side entity that manages the foreign key

The database column name used for storing the primary key

The fetch strategy used for loading the associated entity

The cascade operations applied to the relationship

Q11. What is the primary advantage of using JWT (JSON Web Tokens) for authentication in Spring Boot applications?

JWT tokens can only be decoded by the server, ensuring maximum security.

JWT tokens are stateless, reducing the need to store session information on the server.

JWT tokens provide built-in encryption, ensuring data integrity during transmission.

JWT tokens automatically expire after a certain period, enhancing security.

Q12. Which of the following is NOT a valid way to communicate between a servlet and a client browser?

HTTP response headers

HTTP request parameters

Java Native Interface (JNI)

Session attributes

Q13. Which of the following scenarios is most suitable for implementing database normalization in a Spring Boot application?

When the application requires denormalized data for efficient querying and reporting.

When the application has limited data volume and complexity, making normalization unnecessary.

When the application needs to enforce complex business rules and validation logic at the database level.

When the application involves managing large volumes of data with multiple interrelated entities and relationships.

Q14. What is the purpose of using optional = false in a @OneToOne association in Spring Boot JPA?

It ensures that cascading operations are applied to the associated entity.

It allows the associated entity to be null, making the association optional.

It enforces the eager fetching of the associated entity.

It indicates that the association is mandatory and cannot be null.

Q15. In the context of version control, what is a "commit"?

A request to merge code changes into the main branch

A snapshot of the project's current state at a specific point in time

An action to revert changes made to a file

A notification to team members about code updates

Q16. Consider the following Spring Boot controller method, How can you improve error handling in this method to provide more informative error messages without exposing sensitive information?

```
1 @PostMapping("/products")
2
3 public ResponseEntity<Product> createProduct(@RequestBody Product newProduct) {
4
5     try {
6
7         Product createdProduct = productService.createProduct(newProduct);
8
9         return ResponseEntity.status(HttpStatus.CREATED).body(createdProduct);
10
11     } catch (ProductCreationException e) {
12
13         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(null);
14
15     }
16
17 }
```

Implement centralized logging and auditing mechanisms using AOP and Spring Boot's logging frameworks to capture detailed error logs and context information, facilitating post-mortem analysis and incident response.

Use Spring Boot's built-in `@ResponseStatus` annotation to customize HTTP error responses based on specific exception types, ensuring consistent error handling across all controller methods.

Integrate with Spring Boot Actuator's error endpoint to expose application-wide error metrics and diagnostics, enabling real-time monitoring and troubleshooting of error conditions in production environments.

Implement a custom exception handler using `@ControllerAdvice` to intercept `ProductCreationException` and return a structured error response with detailed error codes and messages, adhering to RESTful API best practices.

Q17. How can you define a checked custom exception in Spring Boot?

Annotations like `@GetMapping` can handle all potential errors

Extend the Exception class

Implement the Throwable interface

Extend the RuntimeException class

Q18. In Spring Framework, what is the purpose of using the `@Qualifier` annotation along with `@Autowired`?

It disables dependency injection for a specific bean

It selects a specific bean when multiple beans of the same type are available

It creates a new bean instance during application startup

It replaces the need for the `@Autowired` annotation

Q19. Which HTTP status code is typically used to indicate that a requested resource is not found in a Spring Boot REST API?

200

403

404

500

Q20. Which ORM design pattern is responsible for tracking changes made to objects and coordinating database updates within a transaction?

Unit of Work pattern

Proxy pattern

Singleton pattern

Factory pattern

Q21. Which of the following is true about the `@JoinColumn` annotation in the context of a one-to-one association in Spring Boot JPA?

It defines the foreign key column in the database table of the owning entity.

It specifies the primary key column in the inverse side of the relationship.

It is used only in unidirectional associations.

It can only be used with `@OneToOne` annotations.

Q22. What is the primary goal of normalization in database design?

To eliminate redundancy and minimize data duplication.

To maximize data integrity and enforce referential constraints.

To optimize query performance and improve data retrieval speed.

To standardize data formats and ensure compatibility across different systems.

Q23. In a Spring Boot REST API application, what does the term "content negotiation" refer to in the context of handling HTTP requests and responses?

It denotes the negotiation of session management and authentication mechanisms between the client and server to establish a trusted and authenticated connection.

It refers to the negotiation of HTTP methods and request headers between the client and server to establish a secure and reliable communication channel.

It signifies the negotiation of caching directives and cache-control policies between the client and server to optimize resource caching and reduce network latency.

It denotes the process of negotiating the content type and encoding between the client and server to determine the most suitable representation of the resource.

Q24. In a servlet container, what is the purpose of the servlet life cycle methods `init()`, `service()`, and `destroy()`?

`init()` creates a new instance of the servlet, `service()` processes HTTP requests, and `destroy()` shuts down the servlet container.

`init()` configures servlet mappings, `service()` dispatches requests to appropriate servlets, and `destroy()` terminates the current session.

`init()` sets up database connections, `service()` executes business logic, and `destroy()` closes database connections.

`init()` initializes the servlet, `service()` handles incoming requests, and `destroy()` releases resources.

Q25. What is the primary benefit of using the `@MapsId` annotation in a one-to-one association with shared primary keys?

It simplifies the configuration by removing the need for a separate foreign key column.

It allows the associated entity to have a composite key.

It ensures that the primary key of the owning entity is automatically set as the primary key of the associated entity.

It improves the performance of queries involving the association.

Q26. What is the simplest way to override the default validation message for a single Bean Validation constraint?

- Specify a custom message using the message attribute in the validation annotation.
- Create a custom validation framework from scratch
- Modify the Hibernate Validator source code
- Disable Bean Validation and handle all validation manually

Q27. In JPQL, what is the significance of the DISTINCT keyword in a SELECT clause?

- It specifies that duplicate entities should be excluded from the query result.
- It indicates that the query should return only entities with distinct property values.
- It defines the criteria for ordering query results based on entity properties.
- It signifies that the query should return a distinct set of entity properties.

Q28. What is the purpose of the SpringApplication class in Spring Boot?

- It provides utilities for loading and configuring the Spring application context.
- It manages the lifecycle of Spring beans in the application context.
- It bootstraps the Spring Boot application and launches the application context.
- It initializes the embedded web server for deploying Spring Boot applications.

Q29. In JPA, which annotation would you use to define a non-persistent field in an entity?

- @Id
- @Transient
- @NonPersistent
- @Column(nullable = false)

Q30. In a Spring Boot application, how can you handle errors and exceptions globally across all controller methods?

By using try-catch blocks within each controller method to handle specific exceptions.

By annotating individual controller methods with `@ExceptionHandler` to handle specific exception types.

By configuring a centralized error handler using `@ControllerAdvice` and `@ExceptionHandler` to intercept and process exceptions globally.

By returning a custom error response directly from the controller method in case of errors.

Q31. What is a drawback of handling exceptions using try-catch blocks in every controller method?

It leads to repetitive code and makes centralized error handling difficult

It improves code readability by reducing duplication

It automatically converts all exceptions into HTTP 404 responses

It eliminates the need for `@ControllerAdvice`

Q32. Which annotation is used to specify the database table name for a JPA entity?

`@NamingStrategy`

`@EntityTable`

`@Table`

`@ColumnNaming`

Q33. In Spring Boot, what is the purpose of the `@RestController` annotation compared to `@Controller`, and how does it affect response handling?

`@RestController` is a specialized annotation for handling asynchronous requests, allowing for non-blocking I/O operations and more efficient resource utilization compared to `@Controller`.

`@RestController` ensures that all methods within the annotated class are treated as REST endpoints, automatically generating Swagger documentation and API documentation based on method signatures.

`@RestController` combines the behavior of `@Controller` and `@ResponseBody`, automatically converting method return values to JSON or XML responses, making it suitable for RESTful APIs.

@RestController is used specifically for controlling the flow of requests and responses in the application, providing finer control over HTTP status codes and response headers compared to @Controller.

Q34. What is the primary purpose of a servlet filter in a Java web application?

To intercept and modify HTTP requests and responses before they reach the servlet or after they leave.

To cache static resources like images and CSS files to improve page loading speed.

To encrypt sensitive data transmitted over HTTP connections.

To manage user sessions and enforce access control policies.

Q35. What is the primary advantage of token-based authentication over traditional session-based authentication?

It is easier to implement

It provides better security against CSRF attacks

It allows stateless communication between the client and server

It offers stronger encryption for user credentials

Q36. Which Spring Security filter processes username/password authentication requests?

UsernamePasswordAuthenticationFilter

SecurityContextPersistenceFilter

ExceptionTranslationFilter

AuthorizationFilter

Q37. What is the purpose of the HAVING clause in JPQL?

- It filters the grouped results based on a specified condition.
- It specifies the criteria for joining multiple entities.
- It defines the order of the query results.
- It performs an aggregation operation on the query results.

Q38. Consider the following JPQL query, What does :deptName represent in below query?

```
1 SELECT e FROM Employee e WHERE e.department.name = :deptName
```

- A derived attribute of the Employee entity class that specifies the department name.
- A static placeholder indicating the position of the department name in the query.
- A named query parameter that will be dynamically replaced with a department name at runtime.
- A predefined parameter representing the name of the Department entity class.

Q39. Consider the following controller method: What happens if OrderCreationException is thrown?

```
1 @PostMapping("/orders")
2 public ResponseEntity<Order> createOrder(@RequestBody Order newOrder) {
3     try {
4         Order createdOrder = orderService.createOrder(newOrder);
5         return ResponseEntity.status(HttpStatus.CREATED).body(createdOrder);
6     } catch (OrderCreationException e) {
7         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(null);
8     }
9 }
```

- The exception is caught, and the method returns HTTP 500 Internal Server Error
- The application automatically retries the request
- The exception is ignored, and HTTP 201 is returned
- The request is redirected to another controller

Q40. When designing a Spring Boot application, what is the primary role of the controller layer in the context of handling HTTP requests and responses?

Executing business logic and enforcing application-specific rules.

Interacting with the database to perform CRUD (Create, Read, Update, Delete) operations.

Validating incoming requests and serializing responses into JSON or XML format.

Handling HTTP request mapping, request parameter parsing, and generating appropriate HTTP responses.

Q41. Which of the following statements accurately describes the role of Tomcat in web development?

Tomcat is a web server that exclusively serves static content like HTML files.

Tomcat is primarily used for database management and does not handle HTTP requests.

Tomcat is a load balancer that distributes incoming requests across multiple server instances.

Tomcat is a servlet container that executes Java servlets and manages JavaServer Pages (JSP).

Q42. What is the difference between authentication and authorization in Spring Security?

Authentication verifies user credentials, while authorization determines what resources a user can access.

Authentication determines what resources a user can access, while authorization verifies user credentials.

Authentication and authorization are synonymous terms in Spring Security.

Authentication and authorization both refer to the process of verifying user credentials.

Q43. In a Spring Boot REST API, what is the significance of using HTTP status code 204?

It indicates that the requested resource has been successfully created on the server.

It signifies that the server has processed the request but requires additional action from the client to complete the operation.

It denotes that the server successfully processed the request but intentionally returned no content in the response body.

It represents a client-side error indicating that the request syntax is incorrect or malformed.

Q44. What is the purpose of the PATCH HTTP method?

To create a new resource on the server.

To update an existing resource with partial modifications.

To delete a resource permanently from the server.

To retrieve metadata about a resource without transferring the entire content.

Q45. In Spring Security, what is the purpose of an AuthenticationProvider?

To authenticate users based on their roles and permissions.

To manage user sessions and cookies.

To intercept and handle authentication requests.

To validate user credentials and authenticate users.

Q46. What is the purpose of the @Version annotation in JPA entities?

It specifies the database version compatibility.

It indicates the version of the entity for optimistic locking.

It marks the entity as eligible for caching.

It defines the entity's lifecycle management strategy.

Q47. If you need to perform a complex query involving a many-to-one relationship, which feature of Spring Data JPA would you use to construct such queries dynamically?

EntityManager

JPQL

Criteria API

Named Queries

Q48. Your Spring Boot API returns a "500 Internal Server Error" for certain requests. What are some debugging strategies to identify the root cause?

Restart the server and retry the request.

Analyze network traffic using a sniffer tool.

Increase the logging level for all classes in the application.

Check server logs for specific error messages.

Q49. What is the purpose of the @JoinTable annotation in Spring Boot JPA?

To specify the join column for a relationship

To define a custom repository method

To create a join table for a many-to-many relationship

To indicate the fetch plan for lazy-loaded entities

Q50. What role do stereotype annotations (@Service, @Repository, etc.) play in Spring Framework applications?

They enforce strict data typing

They provide metadata for dependency injection

They facilitate aspect-oriented programming

They optimize database access patterns

Q51. Consider the following Spring Boot controller method, What are the potential drawbacks of directly returning a list of entities in the response body in a production-grade Spring Boot application?

```
1 @GetMapping("/products")
2 public ResponseEntity<List<Product>> getAllProducts() {
3     List<Product> products = productService.getAllProducts();
4     return ResponseEntity.ok(products);
5 }
```



It exposes internal implementation details of the application, including database schema and entity mappings, potentially posing security risks such as SQL injection attacks.

It may result in inefficient client-side processing and rendering of the response data, leading to slower page load times and degraded user experience.

It may lead to performance degradation and increased network bandwidth consumption due to the size of the response payload, especially for large datasets.

It violates RESTful API principles by tightly coupling the client with the server's internal data model, hindering scalability and flexibility in future API enhancements.

Q52. You have a Spring Boot controller with a `@GetMapping` method that retrieves a list of users. How can you ensure pagination is supported in the request using parameters?

Define a custom annotation for pagination logic.

Implement manual pagination logic within the controller method.

Utilize built-in Spring Data JPA features for pagination.

Pagination is not possible with `@GetMapping`.

Q53. Consider a Customer entity that has a many-to-one relationship with a Country entity. Which of the following is the correct way to define this relationship in the Customer entity if you want to override the default fetch behavior with explicit eager loading?

```
@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "country_id")
private Country country;
```

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "country_id")
private Country country;
```

```
@ManyToOne
@JoinColumn(name = "country_id", updatable = false, insertable = false)
private Country country;
```

```
@ManyToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "country_id")
private Country country;
```

Q54. In a Spring Boot application, which Spring feature is commonly used to manage database transactions declaratively?

Spring Transaction Management using `@Transactional`

Spring MVC DispatcherServlet

Spring Security Filter Chain

Spring Boot Starter Dependency Management

Q55. What is the purpose of HTTP Status Code 202 Accepted?

The request has been accepted for processing, but the processing has not yet been completed

The requested resource has been created successfully

The client must perform additional actions before processing

The requested resource was not found

Q56. In JPA, what does the @ElementCollection annotation signify when used with a field?

It specifies a collection of embedded elements.

It defines an element of a composite primary key.

It marks the field as a collection of entities.

It indicates a collection of simple values or embeddable objects.

Q57. In Spring Boot, what is the purpose of the Actuator module in debugging and monitoring applications?

It provides endpoints for exposing application metrics and status information.

It enables remote debugging of Spring Boot applications.

It automates the generation of bug reports and stack traces.

It integrates with profiling tools for detailed performance analysis.

Q58. Which of the following annotations is used to specify the column name of the foreign key in a one-to-one association in Spring Boot JPA?

@JoinColumn

@Column

@ForeignKey

@ForeignKeyColumn

Q59. When encountering a NullPointerException in a Spring Boot application, what is the recommended approach for debugging?

- Ignoring the exception as it's a common occurrence
- Identifying the root cause by examining the stack trace
- Restarting the application server
- Disabling exception handling to see more detailed error messages

Q60. ORM allows developers to work with which type of data structures?

- Arrays
- Lists
- Maps
- Objects

Q61. Which of the following best describes the process of denormalization in the context of a Spring Boot application?

- Combining multiple normalized tables into a single denormalized table to improve query performance.
- Breaking down a single table into multiple related tables to eliminate redundancy and dependency.
- Introducing redundancy and duplication in a database schema to optimize data retrieval and manipulation.
- Dividing a database schema into multiple related tables to enforce referential integrity constraints.

Q62. When implementing exception handling in a Spring Boot controller, what is the benefit of using @ControllerAdvice combined with @ExceptionHandler annotations?

- They provide a centralized location to define exception handling logic for multiple controllers.
- @ControllerAdvice defines the exception type, and @ExceptionHandler handles the specific error message.
- These annotations are only necessary for custom exceptions.
- They are used for security exception handling specifically.

Q63. What is the primary purpose of the payload in a JWT?

- To store claims (such as user information and token metadata)
- To encrypt the token contents
- To digitally sign the token
- To store only the token's expiration time

Q64. In Spring Security, what is the purpose of the @Order annotation in filter configuration?

- To specify the order in which filters should be applied.
- To control the order of method execution within a filter.
- To indicate the order of execution for request processing.
- To define the order of exception handling in the filter chain.

Q65. What is the role of the service layer in a Spring Boot REST API architecture?

- Managing HTTP requests and routing them to appropriate endpoints.
- Implementing business logic and interacting with repositories for data operations.
- Performing CRUD operations directly on the database.
- Validating request parameters and serializing responses.

Q66. What is Autowiring in Spring Framework?

- It is a feature that enables Spring to dynamically generate bean instances based on runtime conditions.
- It is a design pattern used for manual injection of dependencies in Spring applications.
- It is a mechanism for defining explicit wiring between beans in Spring configuration files.
- It is a feature that allows Spring to automatically detect and inject dependencies into beans.

Q67.

In a Spring Boot JPA application, how do you configure a one-to-one relationship between User and Profile entities where User is the owning side?

By using @OneToOne(mappedBy = "user") annotation in the User entity.

By using @OneToOne annotation in the User entity and @OneToOne(mappedBy = "profile") in the Profile entity.

By using @OneToOne annotation in both User and Profile entities.

By using @OneToOne annotation in the User entity and @OneToOne(mappedBy = "user") in the Profile entity.

Q68. Which normal form is primarily used to remove transitive dependencies and separate attributes that depend on non-key attributes?

First Normal Form (1NF)

Boyce-Codd Normal Form (BCNF)

Second Normal Form (2NF)

Third Normal Form (3NF)

Q69. How do you ensure that deleting an Order entity will also delete its associated OrderItem entities in a one-to-many relationship?

Use @OneToMany(mappedBy = "order", orphanRemoval = true) in Order.

Use @OneToMany(mappedBy = "order", cascade = CascadeType.ALL) in Order.

Use @OneToMany(cascade = CascadeType.REMOVE) in Order.

Use @ManyToOne(cascade = CascadeType.ALL, orphanRemoval = true) in OrderItem.

Q70. What is the purpose of applying the @JsonIgnore annotation to a field in a Spring Boot entity class?

To define a custom JSON property name for the field, allowing for aliasing and mapping between entity properties and JSON attributes in API payloads.

To configure the field as a primary key or unique identifier for the entity, ensuring data integrity and uniqueness constraints are enforced by the database.

To indicate that the field should be ignored during object mapping and persistence operations performed by Spring Boot's ORM (Object-Relational Mapping) framework.

To specify that the field should be excluded from JSON serialization and deserialization processes, preventing sensitive data from being exposed in API responses.

Q71. In Spring Security, which component is responsible for storing the authenticated user's details after successful authentication?

SecurityContext

AuthenticationEntryPoint

AccessDeniedHandler

PasswordEncoder

Q72. Which HTTP status code is typically returned when a user attempts to access a secured endpoint without proper authentication in Spring Security?

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

Q73. When using @Query annotation in Spring Data JPA, which syntax would you use to define a native SQL query?

@Query(value = "SELECT * FROM table_name", nativeQuery = true)

@Query("SELECT * FROM table_name")

@Query(value = "SELECT * FROM table_name")

@NativeQuery("SELECT * FROM table_name")

Q74. When designing a Spring Boot REST API, what does the term "layering" refer to in the context of application architecture?

It refers to the physical separation of server-side and client-side components in a distributed system, ensuring modularity and scalability.

It indicates the process of optimizing network communication and data transfer between different layers of the application, minimizing latency and bandwidth consumption.

It signifies the hierarchical organization of software components based on their relative dependencies and interactions, ensuring encapsulation and abstraction.

It denotes the logical partitioning of application functionality into distinct layers such as presentation, business logic, and data access, promoting separation of concerns and maintainability.

Q75. In Spring Boot, which logging level provides the most detailed information and is commonly used for fine-grained diagnostic tracing, one level below DEBUG?

ERROR

WARN

DEBUG

TRACE

Q76. In a scenario with DI, who is responsible for creating the object dependencies?

The programmer

The IoC container

The object itself

The operating system

Q77. Which of the following is generally NOT considered a benefit of using ORM frameworks compared to JDBC?

Reduced boilerplate database access code

Automatic mapping between objects and database tables

Improved maintainability of data access code

Guaranteed better runtime performance in all scenarios

Q78. Which Spring Boot feature is commonly used to globally customize error responses across multiple controllers?

@ControllerAdvice with @ExceptionHandler

@ResponseBody

@Autowired

@Repository

Q79. In JPQL, what does the GROUP BY clause primarily facilitate?

Filtering query results based on specified conditions.

Joining multiple entities in the query.

Aggregating query results based on common property values.

Sorting query results based on specified criteria.

Q80. What is the purpose of using the fetch attribute in a @OneToOne association in Spring Boot JPA?

It specifies the type of join to be used when fetching associated entities.

It indicates whether the association should be eagerly or lazily fetched.

It defines the fetch plan to be used when loading entities from the database.

It controls the direction of fetching between the owning and inverse sides of the association.

Q81. Consider the following Spring Boot controller method, What is the purpose of the @Valid annotation applied to the User parameter?

```
1 @PostMapping("/users")
2
3 public ResponseEntity<User> createUser(@Valid @RequestBody User newUser) {
4
5     // Logic to create a new user
6
7     return ResponseEntity.status(HttpStatus.CREATED).body(newUser);
8
9 }
```



It indicates that the User object should be validated against predefined constraints specified in the domain model before being processed.

It triggers automatic conversion and mapping of the request body to the User object using Spring Boot's built-in data binding and validation mechanisms.

It specifies the HTTP method to be used for the request mapping, ensuring that only POST requests are accepted for creating new users.

It enables Cross-Origin Resource Sharing (CORS) for the /users endpoint, allowing cross-origin requests from specified origins.

Q82. What is the purpose of the UserDetails interface in Spring Security?

To define the structure of JWT tokens.

To represent the details of a user, including username, password, and authorities.

To manage user sessions and cookies.

To specify access rules for different endpoints.

Q83. When creating a Spring Boot REST API, what is the primary function of the repository layer?

Handling HTTP request/response cycles and managing API endpoint routing.

Executing business logic and enforcing application-specific rules.

Interacting with the database to perform CRUD operations and managing data persistence.

Validating incoming requests and serializing responses into JSON or XML format.

Q84. What is the role of the "Session" object in ORM frameworks like Hibernate?

It represents a connection to the database and manages transactions.

It's responsible for generating entity classes from database tables.

It provides utilities for executing raw SQL queries directly.

It handles object serialization and deserialization.

Q85. In Spring Boot JPA, what does the @Repository annotation indicate?

It marks a class as a Spring MVC controller.

It specifies a bean for database repository operations.

It configures database connection pooling.

It enables asynchronous method execution.

Q86. What is the primary purpose of WSDL (Web Services Description Language) in SOAP-based web services?

To define the interface, operations, and communication details of a web service

To encrypt messages exchanged between web service clients and servers

To store web service authentication credentials securely

To define database schemas used by the web service

Q87. Which annotation enables the automatic scanning of JPA repositories in Spring Boot?

@EnableJpaRepositories

@SpringBootJpa

@AutoScanRepositories

@EnableRepositories

Q88. In Spring Boot JPA, how can you specify a native SQL query with named parameters?

By using numbered parameters (e.g., ?1, ?2)

By using the @NamedNativeQuery annotation

By using the @NamedNativeQuery annotation along with named parameters

By using named parameters (e.g., :param1, :param2)

Q89. In a bidirectional one-to-many relationship, which side is typically the owning side and why?

The parent entity is the owning side because it holds the collection of child entities.

The child entity is the owning side because it contains the foreign key.

Both sides are owning sides to ensure data consistency.

Neither side is the owning side; the relationship is managed through a join table.

Q90. In a Spring Boot application, how are referential integrity constraints typically enforced at the database level?

- Through the use of Spring Data JPA's built-in validation mechanisms.
- By defining foreign key relationships between database tables.
- By implementing custom validation logic in Spring Boot service classes.
- Through the application of Spring Boot's declarative transaction management.

Q91. Which of the following is NOT a stereotype annotation in Spring?

- @Service
- @Repository
- @Component
- @Entity

Q92. What is the benefit of using Spring Actuator in a Spring Boot application?

- To simplify database access and management.
- To provide health checks and monitoring endpoints.
- To enable automatic deployment pipelines.
- To configure logging and exception handling.

Q93. When debugging a Spring Boot application, which logging level is typically used to log critical errors that require immediate attention?

- DEBUG
- WARN
- ERROR
- INFO

Q94. What is the primary responsibility of the Spring IoC Container?

Managing the lifecycle, configuration, and dependencies of Spring beans

Executing SQL queries

Managing HTTP requests

Compiling Java classes

Q95. Which of the following accurately describes the role of a web service in client-server communication?

Web services allow clients to directly access databases on the server.

Web services are only accessible through web browsers and not other client applications.

Web services are primarily used for rendering graphical user interfaces (GUIs) on the client side.

Web services facilitate communication between different applications over a network using standardized protocols.

Q96. In a Spring Boot REST API, which component is responsible for converting Java objects to JSON responses and JSON requests to Java objects?

Service Layer

Spring MVC HTTP Message Converters

Repository Layer

Security Layer

Q97. How does runtime polymorphism support Dependency Injection (DI) in Spring?

It allows different implementations of an interface to be injected and used through a common interface at runtime

It enables Spring to create bean definitions during compilation

It allows Spring to inject dependencies only based on bean names

It prevents multiple implementations of an interface from being injected

Q98. When dealing with exceptions in a Spring Boot REST API, how can you return a custom error response for a specific exception?

By using try-catch blocks in every controller method.

By annotating a method in a `@ControllerAdvice` class with `@ExceptionHandler` for the specific exception type.

By throwing the exception from the service layer directly.

By returning an error code from the repository layer.

Q99. What is the purpose of Spring Boot in the context of Spring Framework?

To optimize the performance of Spring applications.

To provide additional features not available in the core Spring Framework.

To replace the traditional Spring MVC framework.

To simplify the configuration and deployment of Spring applications.

Q100. Consider the following JPA entity using Java 17+. Which modern Java type should be used instead of `java.util.Date` for storing a person's date of birth?

```
1 @Entity
2 public class Person {
3
4     @Id
5     private Long id;
6
7     private String name;
8
9     // Which modern type should replace this field's type?
10    private Date dob;
11
12    // Getters and setters
13 }
```

`java.sql.Timestamp`

`java.time.LocalDate`

`java.util.Calendar`

`java.lang.String`

Q101. Hannah is developing a Spring Boot application where he needs to design an API endpoint that validates and handles nested JSON objects with complex structures. She needs to ensure that validation is thorough and properly managed. How would you design an API endpoint that needs to validate and handle nested JSON objects with complex structures?

Implement manual validation logic in the controller method.

Use nested `@Valid` annotations in the request body and define appropriate validation constraints.

Configure nested validation in application.properties.

Use different endpoints for each level of the nested structure.

Q102. What will be the outcome if a method annotated with `@PostConstruct` throws an exception during bean initialization?

The method will be retried until it succeeds

The exception will be ignored, and the method will continue

The exception will be logged, and the bean initialization will proceed

The Spring context will fail to start, and the bean will not be initialized

Q103. If you need to activate a Spring component only when a specific system property is set to true, which annotation is most appropriate?

`@ConditionalOnClass`

`@ConditionalOnBean`

`@ConditionalOnProperty`

`@ConditionalOnMissingBean`

Q104. Which listener is activated when a new attribute is added to the `ServletContext`?

`ServletContextAttributeListener`

`ServletContextListener`

`HttpSessionListener`

`ServletContextEventListener`

Q105. In a Spring Boot application, what will be the result of a GET request if a method is annotated with `@GetMapping("/users/{id}")` and the method parameter is annotated with `@PathVariable`?

The value of `{id}` is mapped to a request header

The URL parameter is converted to a query parameter

The method will ignore the URL parameter

The value of {id} in the URL is injected into the method parameter

Q106. When caching is enabled, which response header controls how long the client can cache the response?

Content-Length

ETag

Transfer-Encoding

Cache-Control

Q107. In a scenario where you need to configure different database sources for read and write operations, which annotations would you use to mark methods that interact with read and write databases respectively?

@ReadOnly and @WriteOnly

@Transactional(readOnly = true) and @Transactional(readOnly = false)

@ReadTransaction and @WriteTransaction

@DataSourceRead and @DataSourceWrite

Q108. Which of the following SQL statements is used to perform an inner join?

SELECT * FROM table1 LEFT JOIN table2 ON table1.id = table2.id;

SELECT * FROM table1 RIGHT JOIN table2 ON table1.id = table2.id;

SELECT * FROM table1 JOIN table2 ON table1.id = table2.id;

SELECT * FROM table1 FULL JOIN table2 ON table1.id = table2.id;

Q109. What is the best approach to maintain backward compatibility when versioning REST API endpoints in Spring Boot?

Use URL path versioning (e.g., /api/v1/resource) and maintain separate controllers for each version.

Configure versioning in application.properties and handle it in the controller.

Implement a versioning strategy in the service layer.

Implement separate endpoints for different combinations of search parameters.

Q110. Steve wants to implement a REST endpoint for real-time updates like stock prices. What is the best approach to achieve real-time communication with clients?

- Use WebSockets or Server-Sent Events (SSE) to push updates to clients.
- Implement polling in the client to periodically check for updates.
- Use long polling techniques in the REST endpoint.
- Configure a background task to push updates to clients.

Q111. To enable handling of POST requests with JSON payloads in Spring Boot, what configuration is necessary?

- By configuring `HttpMessageConverter` beans
- By setting `Content-Type` headers manually
- By using `@RequestParam` for JSON fields
- By defining custom request handlers

Q112. What is the default role prefix in Spring Security?

- USER_
- AUTH_
- SEC_
- ROLE_

Q113. Given the annotation `@ComponentScan(basePackages = "com.example")`, what does the `basePackages` attribute achieve?

- To specify the base packages for scanning Spring beans
- To set the base package for component scanning in tests
- To define the packages where Spring configuration classes are located
- To restrict component scanning to specific sub-packages

Q114. How does Spring Boot handle request data binding for a POST request with a complex object as the request body?

- It requires manual parsing of request data

It uses @RequestBody to bind JSON or XML data to the complex object

It binds form parameters to the complex object fields

It uses @PathVariable for binding complex objects

Q115. Which algorithm is commonly used to sign a JWT token?

MD5

HS256

SHA-256

RSA

Q116. Which class is commonly used to configure security settings in a Spring Boot application?

WebSecurityConfig

SecurityAdapter

SecurityConfiguration

WebSecurityConfigurerAdapter

Q117. How would you manage conditional requests where the response should vary based on a client's preferred language?

Implement separate endpoints for different languages.

Use a custom exception handler for language-specific responses.

Use @RequestHeader to read the Accept-Language header and adjust the response.

Configure language preferences in application.properties.

Q118. What is the purpose of the @Produces annotation in JAX-RS?

Configures the endpoint URL

Sets the content type of incoming requests

Indicates the HTTP verb to use

Specifies the MIME type that a method returns

Q119. In a web application scenario where a developer needs to request that the connection be switched to a different protocol (e.g., WebSocket or HTTP/2), which header field should they use?

X-HTTP-Version

Protocol-Version

Protocol

Upgrade

Q120. Consider the following SQL query:
What type of result does this query produce?

```
1 SELECT Students.name, Courses.course
2 FROM Students
3 FULL JOIN Enrollments ON Students.id = Enrollments.student_id
4 FULL JOIN Courses ON Enrollments.course_id = Courses.id;
5 |
```

Students and their courses, including those with no enrollments or courses.

Only students with enrollments and their courses.

Only courses with enrolled students.

A cross join of all students and courses.

Q121. How do clients and servers communicate in a Webservice environment?

Through direct connections using TCP/IP.

By exchanging messages in a predefined format over HTTP.

Using proprietary protocols specific to each application.

Using proprietary protocols general to all application.

Q122. What is a servlet in the context of Webservices?

A small program that runs on a client and interacts with a server.

A Java program that extends the functionality of a web server by handling client requests.

A database that stores information about users and their permissions.

A protocol used for secure communication between clients and servers.

Q123. What is the role of a Tomcat server in a Webservice environment?

To provide a graphical user interface for web applications.

To store and manage large amounts of data.

To host and execute Java servlets and JSPs.

To manage user authentication and authorization.

Q124. What is the main advantage of using JSON over XML for data exchange in Webservices?

JSON is more secure than XML.

JSON is more efficient and lightweight than XML.

JSON is a more widely used standard than XML.

JSON is easier to understand and implement than XML.

Q125. What is the primary difference between XML and JSON?

XML is a markup language while JSON is a scripting language.

XML is used for data exchange while JSON is used for web page design.

XML is a markup language while JSON is a lightweight data-interchange format.

XML is more efficient than JSON for data exchange.

Q126. Which of the following is a valid JSON data structure?

`{ "name": "John", "age": 30 }`

John30

["John", 30]

"name": "John", "age": 30

Q127. Which of the following is NOT a benefit of using Webservices?

Increased interoperability between different applications.

Reduced development time and cost.

Improved security and reliability.

Increased dependence on proprietary protocols.

Q128. How does Hibernate implement persistence?

By using a separate database connection for each object.

By storing objects in memory and synchronizing them with the database periodically.

By using a single database connection for all objects.

By converting objects into SQL statements and executing them against the database.

Q129. What is the difference between JPA and Hibernate?

JPA is a specific ORM implementation, while Hibernate is a standard specification.

Hibernate is a specific ORM implementation, while JPA is a standard specification.

JPA is a database management system, while Hibernate is an ORM tool.

Hibernate is a programming language, while JPA is a framework.

Q130. What is the purpose of the @GeneratedValue annotation in JPA?

To automatically generate values for the primary key.

To specify the database connection details.

To map an object property to a database column.

To create a new database table automatically.

Q131. How does an ORM tool handle data relationships between objects?

By using separate database tables for each object.

By using foreign keys to link related objects in the database.

By storing all related objects in a single table.

By using a separate database connection for each relationship.

Q132. What is the purpose of the @Entity annotation in JPA?

To define the primary key of an entity.

To map an object class to a database table.

To specify the database connection details.

To create a new database table automatically.

Q133. Which of the following is a valid JPA query language?

SQL

JPQL

MYSQL

MSSQL

Q134. Which of the following is a common method for deleting data from a database using JPA?

Using a SQL delete statement.

Using the EntityManager.persist()

Using the EntityManager.remove()

Using the EntityManager.merge()

Q135. What is the purpose of the @Id annotation in JPA?

To define the primary key of an entity.

To map an object property to a database column.

To specify the database connection details.

To create a new database table automatically.

Q136. Which of the following is a common method for retrieving data from a database using JPA?

Using a SQL query.

Using a JPQL query.

Using a Hibernate query language (HQL

All of the above.

Q137. Which of the following is a common method for updating data in a database using JPA?

Using a SQL update statement.

Using the EntityManager.merge()

Using the EntityManager.persist()

Using the EntityManager.remove()

Q138. What is the purpose of the @ManyToOne annotation in JPA?

To define a one-to-many relationship between entities.

To define a many-to-many relationship between entities.

To define a one-to-one relationship between entities.

To define a many-to-one relationship between entities.

Q139. What is the primary goal of normalization in relational database design?

To reduce data redundancy and improve data integrity.

To increase the speed of data retrieval.

To simplify the database schema for easier maintenance.

To ensure data consistency across multiple databases.

Q140. Which of the following is NOT a benefit of using a relational database management system (RDBMS)?

Improved data consistency.

Increased data security.

Reduced data redundancy.

Simplified data access for users.

Q141. Which of the following is a disadvantage of normalization?

Increased data redundancy.

Increased complexity of database design.

Reduced data retrieval speed.

Increased risk of data inconsistency.

Q142. Which normal form eliminates repeating groups of data within a table?

First Normal Form (1NF)

Second Normal Form (2NF)

Third Normal Form (3NF)

Boyce-Codd Normal Form (BCNF)

Q143. Which of the following is a common technique used to denormalize a database?

Adding redundant data to improve performance.

Removing unnecessary columns from tables.

Creating new tables to store related data.

Enforcing referential integrity constraints.

Q144. Which of the following is NOT a characteristic of a relational database management system (RDBMS)?

- Data is stored in tables with rows and columns.
- Data is accessed through a structured query language (SQL).
- Data is organized hierarchically in a tree-like structure.
- Relationships between tables are defined through foreign keys.

Q145. Which of the following is an example of a referential integrity constraint?

- A foreign key in one table must match a primary key in another table.
- A column must contain only unique values.
- A column must be a specific data type.
- A column must not be empty.

Q146. A table is in 2NF if it is in 1NF and:

- All non-key attributes are dependent on the primary key.
- All non-key attributes are dependent on a candidate key.
- No non-key attribute is dependent on another non-key attribute.
- The table has only one primary key.

Q147. What does referential integrity ensure in a relational database?

- That all data in the database is accurate and consistent.
- That relationships between tables are correctly enforced.
- That all data is stored in a normalized form.
- That the database is secure from unauthorized access.

Q148. Which of the following is a characteristic of a table in 3NF?

- It has no partial dependencies.

It has no transitive dependencies.

It has no multi-valued dependencies.

All of the above.

Q149. What is the primary function of Git in the context of software development?

Hosting and managing code repositories.

Providing a graphical user interface for interacting with code repositories.

Tracking changes to files and allowing for collaboration on projects.

Automating the deployment of code to production servers.

Q150. Which of the following is NOT a benefit of using a version control system?

Tracking changes made to files over time.

Collaborating on projects with multiple people simultaneously.

Restoring previous versions of files in case of accidental deletion or corruption.

Preventing accidental changes to files by locking them for editing.

Q151. What is the purpose of a "commit" in Git?

To create a new branch of the project.

To merge changes from one branch into another.

To save a snapshot of the current state of the project's files.

To revert changes made to the project back to a previous version.

Q152. Which of the following is a key difference between Git and GitHub?

Git is a distributed version control system, while GitHub is a centralized one.

Git is a command-line tool, while GitHub is a web-based platform.

Git is used for tracking changes to files, while GitHub is used for managing project issues.

Git is used for collaborating on projects, while GitHub is used for hosting code repositories.

Q153. Which Hibernate strategy determines how collections are fetched?

Fetch mode

Fetch type

Fetch plan

Fetch group

Q154. Why might you use a named query instead of a dynamic query in JPQL?

Named queries can only be used with native SQL

Dynamic queries cannot use joins

Named queries support parameter substitution

Named queries are faster as they are precompiled

Q155. John is implementing a service that renders different responses based on the request content type. What method should he override in his Servlet to achieve this?

doGet() and doPost() together.

The init() method to define behavior for various content types.

The service() method for handling all types of requests.

The destroy() method to clean up after requests.

Q156. A web application needs to serve different content types based on the user's device and preferences. How can the application determine the preferred format for response content?

By using the Accept header in the HTTP request.

By implementing URL routing for each device type.

By defining a default content type in the server configuration.

By analyzing the User-Agent string in the request.

Q157. What happens if you omit the cascade attribute in a @OneToMany association?

All changes to the parent are automatically propagated to the child

Changes to the child entities are ignored by JPA

Child entities are not affected by parent operations

JPA will throw an error

Q158. Kiran is designing a new service that requires two different types of notifications: email and SMS. He wishes to implement dependency injection for better code management. Which type of dependency injection is best suited for this scenario when both notifications are required components?

Setter injection for both notifications

Interface injection for the email notification only

Constructor injection for both notifications

No injection is needed; instantiate directly

Q159. You need to calculate the total salary of employees grouped by department using JPQL. Which query would achieve this?

```
SELECT d.name, SUM(e.salary) FROM Department d JOIN d.employees e  
GROUP BY d.name
```

```
SELECT e.salary, COUNT(e) FROM Employee e WHERE e.department IS NOT  
NULL
```

```
SELECT SUM(e.salary), d.name FROM Employee e GROUP BY e.department
```

```
SELECT e.salary, e.department FROM Employee e GROUP BY e.department
```

Q160. What is a functional dependency in database normalization?

A dependency between rows in the same table

A mapping of keys to foreign keys

A dependency between tables in a relationship

A condition where one attribute uniquely determines another

Q161. You have a Spring Boot application with a CommandLineRunner bean. What happens if you define multiple CommandLineRunner beans?

Only the first one gets executed

Spring throws an exception

All CommandLineRunner beans are executed in no specific order

All CommandLineRunner beans are executed sequentially based on their @Order annotation

Q162. What will be the output of the following REST Controller method when called with a valid item ID?

```
1 @RestController
2 @RequestMapping("/api/items")
3 public class ItemController {
4     @Autowired
5     private ItemService itemService;
6
7     @GetMapping("/{id}")
8     public ResponseEntity<Item> getItem(@PathVariable Long id) {
9         Item item = itemService.findById(id);
10        return item != null ? ResponseEntity.ok(item) : ResponseEntity.notFound().build();
11    }
12 }
13 |
```

200 OK with the item details

404 Not Found

500 Internal Server Error

Throws ItemNotFoundException

Q163. A team is implementing a user management system and requires a method in their repository to find a user by their email address. Which repository method naming convention should they use for this purpose?

getEmailUser(String email)

findUserByEmail(String email)

loadUserByEmail(String email)

queryUserByEmail(String email)

Q164. A developer is working on an e-commerce platform and needs to establish a relationship where a `Customer` can place multiple `Orders`, but each `Order` is associated with only one `Customer`. Which Spring Data JPA annotation should the developer use to appropriately model this relationship?

@OneToOne

@ManyToMany

@OneToMany

@ManyToOne

Q165. You configure Spring Security with basic authentication. How can you ensure that all users must log in before accessing any endpoint?

Use `http.authorizeRequests().anyRequest().authenticated()` in the security configuration

Set `spring.security.enable-all=true` in application properties

Enable the `@PreAuthorize` annotation globally

Add `security.require-login=true` in the security filter

Q166. A developer wants to create dynamic queries based on various filter criteria that may change at runtime. What is the best way to achieve this using Spring Data JPA's Specifications?

Use native SQL queries for flexibility.

Implement `CriteriaBuilder` directly for the queries.

Create a repository method without using Specifications.

Implement the `Specification` interface for building queries.

Q167. What do merge conflicts typically indicate during a Git merge?

Merge conflicts occur when changes in different branches affect the same line of code, requiring manual resolution.

Merge conflicts indicate that the repository is corrupted and cannot be used.

Merge conflicts arise from pushing changes to the remote repository.

Merge conflicts happen when branches are merged too frequently.

Q168. A team of developers is implementing an authentication mechanism using JWT in their Spring Boot application. After logging in, the application issues a token with a short expiration time to enhance security. However, a requirement arises where the user session must remain seamless, even when the token expires. What should the development team implement to satisfy this requirement?

Automatically refresh the token upon each request without user intervention.

Prompt the user to log in again every time the token expires.

Implement a refresh token mechanism to obtain a new access token without re-authentication.

Use a long-lived token that never expires.

Q169. A development team is designing a RESTful API using Spring Boot that will leverage JWT for authentication. They decide to include user roles as claims in the JWT tokens. However, they are unsure about how to handle authorization checks efficiently. What is the recommended approach for checking user roles when processing requests?

Decode the JWT token on every request and compare roles in memory.

Store roles in a database and query it every time to authorize access.

Implement a role-based access control mechanism by validating roles decoded from the JWT at request time.

Rely solely on client-side checks for user roles based on token data.

Q170. During a code review, Tom discovers that the JWT implementation includes claims such as `iss`, `sub`, and a custom claim `userRole`. He wants to determine how these claims should be securely handled within the system. What would be Tom's best approach to ensure the JWT claims are appropriately validated?

Check the integrity of claims against trusted server-side data before processing requests.

Validate claims only during the creation of the token, ignoring them during validation.

Avoid using claims as they may expose sensitive information.

Hardcode the expected values of claims in the client-side code for verification.

Q171. Which of these can prevent CSRF attacks in a Spring Boot application?

Rely on basic authentication instead of tokens

Use only GET methods for endpoints

Disable csrf() in HttpSecurity

Use a CSRF token and include it in all state-changing requests

Q172. A company is developing a system with a complex hierarchy of employee roles using Hibernate. They want to ensure that the database architecture efficiently supports querying and ensures minimal redundancy. What inheritance mapping strategy should they implement in their Spring Boot application for optimal performance?

Joined inheritance mapping to separate attributes into relevant tables while sharing common attributes.

Single table inheritance to consolidate all employee role attributes into one table for easier queries.

Table per class strategy to create separate tables for every employee role, allowing full customization.

A non-mapped strategy, relying on an application layer to manage the role hierarchy.

Q173. Which annotation is used to define a class as a Spring-managed bean?

@Bean

@Primary

@Component

@Configuration

Q174. A company is developing a REST API to serve data to a front-end application. They want to ensure that their API is well-structured and adheres to REST principles. What should they use in Spring to define their API endpoints effectively?

Use the @RestController along with @RequestMapping for each endpoint.

Map all requests to a single controller that handles different endpoints.

Use @Controller without REST-specific annotations.

Define API endpoints directly in the service layer.

Q175. What does the `@Entity` annotation indicate in a Spring Data JPA application?

The @Entity annotation specifies the data format for JSON responses.

The @Entity annotation is used for securing data against unauthorized access.

The @Entity annotation allows classes to inherit from other classes.

The @Entity annotation marks a class as a persistent entity for database mapping.

Q176. A company is creating a data warehouse that aggregates data from multiple independent sources for complex bi-annual reporting. The diverse types of data and the necessity for rapid access pose significant challenges. The data engineering team is considering various normalization strategies to balance speed and data redundancy. What should the team prioritize in their approach to ensure fast reporting at the cost of some data integrity?

Normalization to Third Normal Form (3NF) to ensure no transitive dependencies

Denormalization, including redundant data, to improve read performance for analytical queries

Strict adherence to Boyce-Codd Normal Form (BCNF) for all data structures

Using separate, fully normalized databases to maintain data integrity across the system

Q177. What does the @Repository annotation in Spring indicate?

It is a controller for database operations

It is a component for handling REST requests

It is a DAO that interacts with the database

It is used to create a database

Q178. John wants to save a new category entity to his database using Spring Data JPA. Which code snippet should he use?

```
repository.persist(category)
```

repository.insert(category)

repository.save(category)

repository.add(category)

Q179. Consider the following code snippet for a Spring JPA Entity. What will be the output of calling `repository.count()` if five entities are stored in the database?

```
1 @Query("SELECT COUNT(u) FROM User u")
2 long count();
```

0

5

1

An exception will be thrown

Q180. Which annotation is used to define a named query in JPA?

@Query

@NamedQuery

@CustomQuery

@JPQL

Q181. Emma is tasked with retrieving users by their last names in her application using Spring Data JPA. Which method naming convention would she use to create this query in her UserRepository?

findByLastName

searchByLastName

getLastName

retrieveLastName

Q182. Alex is working on a collaborative project involving multiple developers. They are implementing a new feature that requires a significant code update. To ensure that their work does not disrupt the main project, Alex decides to create a new branch for their

updates. What is the most appropriate action Alex should take to effectively manage this situation?

- Merge the feature branch with the main branch before completing the feature.
- Create a branch named "feature-update" and switch to it for making all changes.
- Pull requests should be submitted without creating a separate branch.
- Commit all changes directly to the main branch to keep the history clean.

Q183. What is the primary benefit of using a Spring Data JPA repository?

- It simplifies data visualization in applications.
- It allows for easy implementation of CRUD operations without boilerplate code.
- It restricts all database operations to read-only permissions.
- It manages application-level caching efficiently.

Q184. Tom and his team are developing a web application on GitHub. Each team member needs to work on a different feature concurrently. What GitHub feature should they utilize to manage their work without conflicts?

- Forks
- Releases
- Branches
- Commits

Q185. Lisa wants to secure her methods within a Spring application based on user roles. Which annotation should she use to enforce this requirement?

- @EnableWebSecurity
- @Component
- @PreAuthorize
- @Transactional

Q186. What is the primary role of the @Component annotation in Spring?
A. To create a configuration class for Spring beans

- B. To mark a class as a Spring-managed bean for dependency injection
- C. To define a specific type of Spring bean, like a service or controller
- D. To enable transaction management for a Spring bean

B only

A and B

B and C

D only

Q187. What is the most commonly used data format for exchanging data in Webservices?

XML and JSON

HTML and XML

TXT and CSS

SQL and CSV

Q188. A development team is working on a Spring Boot application that needs to manage configuration properties for different environments (e.g., development, testing, production). They want to avoid hard-coding these properties and prefer a more dynamic way to handle them. What would be the best practice for managing these configurations in Spring Boot?

Define properties in individual Java classes for each environment.

Store properties directly in the main application class.

Use the application.properties or application.yml file with profile-specific configurations.

Rely solely on environment variable settings for all configurations.

Q189. Emily is working on a software project using GitHub. She wants to keep a history of all changes made to the project files. What is the primary benefit of using a version control system like GitHub in this scenario?

It allows unlimited storage for all project files.

It enables tracking changes to files over time.

It automatically backs up all system files.

It restricts file access to specific users.

Q190. John is tasked with improving a customer orders database that currently has duplicate customer information repeated across order rows. Which action should he take first to address the redundancy?

- Create a backup of the database
- Add more fields to the existing table
- Remove duplicate records from the table
- Normalize the database by moving repeating customer data into its own table

Q191. What is the default fetch type for @OneToMany relationships?

- LAZY
- EAGER
- AUTO
- NONE

Q192. In an e-commerce platform, you need to optimize the loading of product data for a category page that frequently changes and has a large number of associated products. What fetch strategy would be most advantageous in balancing performance and data loading efficiency?

- Use EAGER fetching for product categories while using LAZY fetching for associated products to reduce the initial fetch time.
- Use LAZY fetching for both categories and their associated products, minimizing upfront data load but causing multiple queries later.
- Use EAGER fetching for both categories and products to ensure all data is loaded in a single query at the outset.
- Use a custom fetch join query only for frequently accessed product categories and ignore performance considerations for products.

Q193. In a university management system: each student can enroll in multiple courses and each course can have multiple students; each course is taught by a single instructor. Which JPA mapping strategy best fits these requirements?

- Use @ManyToMany between students and courses, and @ManyToOne from Course to Instructor
- Use @OneToMany for students to courses and @ManyToOne for courses to instructors

Use @ManyToMany for students to courses, and @OneToMany for courses to instructors

Use @ManyToOne for both students to courses and courses to instructors

Q194. Sarah is tasked with implementing user authentication in a Spring application. She needs to restrict access to certain endpoints based on user roles. What would be the most effective way for her to set up security within her application?

Implement login logic directly in the controller methods for each endpoint.

Create a configuration class and use @EnableWebSecurity along with configure() methods.

Add security annotations on all service methods for role-checking.

Use JavaScript to control access to backend APIs.

Q195. In Spring Data JPA, what is the purpose of the mappedBy attribute in a bidirectional relationship?

It specifies the column name in the database

It is placed on the inverse (non-owning) side to tell JPA that the foreign key is managed by the field named on the other entity

It is used only to create a one-to-one relationship between entities

It triggers eager loading of the related entity

Q196. Emma has been working on a coding project and wants to save her latest changes with a descriptive message. Which git command should she use to record these changes in her local repository?

git clone

git commit

git push

git pull

Q197. Emily is tasked with creating a configuration class in her Spring Boot application. Which annotation should she use to indicate that her class provides Spring configuration?

@SpringBootConfiguration

@Component

@Configuration

@EnableAutoConfiguration

Q198. Ryan is organizing a library database and wants to make sure that each book entry does not duplicate author information. What process should he consider applying to the database?

Data aggregation

Data normalization

Data migration

Data denormalization

Q199. Alex is working in a team to develop a new application and needs to add a new feature without disrupting the main project. What is the best approach for him to follow using GitHub?

Directly edit the main branch to make the changes.

Create a new branch for the feature and work on it independently.

Delete the main branch and start a new repository.

Share his changes in a chat before making any updates on GitHub.

Q200. What is the main role of entity classes in Spring Boot JPA Mapping?

To define service logic separately from database operations.

To represent tables in a relational database.

To authenticate users in the application.

To handle HTTP requests and responses.

Q201. A development team is working on a collaborative project using Git. They have a feature branch for a new functionality, but another team member has made critical updates to the main branch that should be incorporated into the feature branch before merging it back. What should the developer do to synchronize their feature branch with the latest changes from the main branch effectively?

Directly merge the main branch changes into the feature branch without reviewing the changes.

Create a new branch from the feature branch and cherry-pick the necessary commits from the main branch.

Pull the main branch into the feature branch and resolve any merge conflicts that arise, ensuring integration is complete before merging into the main branch.

Delete the feature branch and create a new one based off the updated main branch to avoid any issues during merging.

Q202. A development team is working on a new feature branch and wants to ensure that their changes are reviewed by the team before merging into the main branch. How should they proceed using GitHub to facilitate this process effectively?

Directly push changes to the main branch without a review.

Open a pull request against the main branch to initiate the code review process.

Create a new branch with the same name as the main branch and push changes there.

Merge the feature branch into the main branch and notify the team via email for review.

Q203. In Spring Data JPA, which annotation is used to define a custom query method in a repository interface?

@Query

@CustomQuery

@NativeQuery

@RepositoryQuery

Q204. A healthcare application is implementing a feature to search for patients based on various dynamic criteria, such as age, condition, and visit date. The application needs to build sophisticated filtering options that can be combined based on user input. Which approach using Spring Data JPA would be most effective in this context?

Use method naming conventions in the repository interface to create specific query methods for each filter.

Create a Specification that combines different criteria using predicates for a flexible query construction.

Implement a series of static query methods in the service layer that checks each filter separately.

Rely on a single SQL query that does not leverage the capabilities of JPA for dynamic filtering.

Q205. When normalizing a relation `Library(BookID, Title, AuthorID, PublisherID, PublisherName)` with `PublisherID` leading to `PublisherName`, what will be the output of the following code snippet intended to extract necessary data while adhering to

normalization rules?

- 1 SELECT BookID, Title, AuthorID
- 2 FROM Library
- 3 WHERE PublisherID IS NOT NULL;

A complete set of books with their respective publisher names.

A list of books, including their author IDs but excluding publisher details.

A combination of books and publishers resulting in unwanted duplication.

An error due to the selection of non-grouped columns.

Q206. What distinguishes JPQL (Java Persistence Query Language) from standard SQL?

JPQL cannot be used with joins.

JPQL is specifically designed to work with Java objects instead of database tables.

JPQL only supports SELECT queries.

JPQL is a programming language on its own.

Q207. Which JPA annotation is used to mark a field as embeddable, allowing its properties to be inlined into the owning entity's table?

@Entity

@Embeddable

@Table

@MappedSuperclass

Q208. Sophia wants to experiment with a new feature for her application without affecting the main codebase. Which git practice is best suited for this purpose?

Deleting the repository

Creating a branch

Committing changes directly to main

Cloning the repository

- Q209. Which of the following is true about the @GetMapping and @PostMapping annotations in Spring?
- A. @GetMapping is used for handling HTTP GET requests, while @PostMapping is used for handling HTTP POST requests.
 - B. @GetMapping is used for sending data to the server, while @PostMapping is used for retrieving data from the server.
 - C. Both @GetMapping and @PostMapping are used for updating data on the server.
 - D. @GetMapping is used for handling HTTP POST requests, while @PostMapping is used for handling HTTP GET requests.

A only

B only

C only

D only

- Q210. A developer is using a Spring Data JPA repository to fetch a list of products from a database. The products need to be sorted by their price in ascending order and filtered to include only those that are currently available. Which JPQL query is most appropriate to implement this requirement in a repository method?

```
@Query("SELECT p FROM Product p WHERE p.available = true ORDER BY  
p.price ASC")  
List<Product> findAvailableProductsSortedByPrice();
```

```
@Query("SELECT p FROM Product p ORDER BY p.price ASC WHERE  
p.available = true")  
List<Product> findAvailableProductsSortedByPrice();
```

```
@Query("FROM Product p WHERE p.available = true SORT BY p.price")  
List<Product> findAvailableProductsSortedByPrice();
```

```
@Query("SELECT p FROM Product WHERE p.price > 0 AND available = true  
ORDER BY p.price ASC")  
List<Product> findAvailableProductsSortedByPrice();
```

- Q211. Tom has a requirement to restrict access to certain features of his application based on user roles. What does Spring Security utilize to manage these access levels?

Roles define access levels for users.

Roles define the password complexity required.

Roles are irrelevant in SQL Injection scenarios.

Roles define how data is stored in the database.

- Q212. A team of developers is working on a large-scale application that requires coordination to prevent conflicts and maintain stability while new features are developed. They have decided to use Git for version control. What branching strategy should they

implement to best handle multiple feature branches while ensuring a stable production environment and minimizing merge conflicts?

- Use a single branch for all development, committing directly to the master branch.
- Create a new branch for every commit made, leading to excessive branching.
- Implement Git Flow strategy, creating dedicated branches for feature, develop, and master.
- Only merge feature branches when releasing, disregarding integration during development.

Q213. Emma is tasked with retrieving all products that are out of stock from the database. Which method signature should she implement in her repository interface to accomplish this?

- findOutOfStock()
- fetchByStockIsZero()
- findByStockEquals(0)
- findByStockLessThan(1)

Q214. Alex discovers a bug while using the application his team developed on GitHub. He wants to track this bug along with other tasks and enhancements. Which GitHub feature should he use to report this issue effectively?

- Wiki
- Commit
- Pull request
- Issue

Q215. John is developing a Spring Boot application and needs to manage dependencies effectively. Which component should he use to simplify the dependency management process?

- ☐ @EnableAutoConfiguration
- ☐ Spring Boot Starter
- ☐ @RestController
- ☐ @Configuration

Q216. What is the primary role of a repository interface in Spring Data JPA?

- ☐ A repository interface manages application configurations.
- ☐ A repository interface simplifies data access by providing CRUD operations automatically.
- ☐ A repository interface handles only read operations for data.
- ☐ A repository interface is used for validating user input in web applications.

Q217. What is the main purpose of the @Autowired annotation in Spring?
A. To configure the Spring bean lifecycle
B. To inject dependencies automatically into Spring beans
C. To enable transaction management in Spring
D. To define a Spring bean

- ☐ A and C
- ☐ B only
- ☐ D only
- ☐ A and B

Q218. Given the JPQL query SELECT e FROM Employee e WHERE e.salary > :minSalary, how is the parameter minSalary passed in a Spring Data JPA repository?

- ☐ By defining it in the method name
- ☐ Using the @Param annotation with a method parameter.
- ☐ Through a separate configuration file
- ☐ By directly binding it to the EntityManager.

Q219. What is JPQL?

Java Persistence Query Language

Java Programming Query Language

Java Protocol Query Language

Java Process Query Language

Q220. Michael is using Spring Data JPA in his project to simplify data access operations. He is particularly interested in finding a way to easily create and interact with repositories. What is the primary advantage of using Spring Data JPA?

It provides an easy way to implement data access layers using repositories.

It requires manual SQL query writing.

It is only compatible with NoSQL databases.

It does not support transactions.

Q221. Ava is optimizing her application and is considering how data is fetched from the database. What is a key distinction about eager fetching in Spring Data JPA?

Eager fetching retrieves data on demand

Lazy fetching retrieves all data immediately

Eager fetching retrieves data immediately.

No difference between fetch strategies.

Q222. Which of the following statements best describes the concept of Dependency Injection (DI) in Spring?

A. DI is a design pattern that allows an object to receive its dependencies from an external source rather than creating them itself.

B. DI is used to manage the lifecycle of beans in the Spring container.

C. DI is used to map the data between the database and Java objects.

D. DI is a Spring feature used for transaction management

A only

A and B

B only

A and C

- Q223. Which of the following is a key feature of RESTful web services?
- A. Uses SOAP protocol
 - B. Stateless architecture
 - C. Requires a specific programming language
 - D. Always returns data in XML format

A and B

B only

B and D

C only

- Q224. Rachel is working on a product inventory database and wants to ensure that her design minimizes data duplication and maintains consistency. What would be the primary advantage of normalizing her database?

It speeds up data retrieval

It reduces data redundancy and improves data integrity

It prevents all user errors

It combines multiple datasets into one

- Q225. Lisa is writing a SQL query to retrieve the names of all products in a store's inventory. Which command will she use to access this information from the database?

GET

SELECT

FETCH

DISPLAY

- Q226. Mark needs to query the database to find all employees whose salary exceeds a specified value. Which JPQL query format should he use?

SELECT * FROM Employee WHERE salary > :salary

SELECT e FROM Employee e WHERE e.salary > :salary

GET e WHERE e.salary > :salary

FIND e BY salary > :salary

Q227. Which Spring Boot annotation is used to define a RESTful controller that handles HTTP requests?

@Service

@Repository

@RestController

Controller

Q228. Which statement best describes the difference between SOAP and RESTful web services?

RESTful services use XML exclusively, while SOAP can use JSON.

SOAP is stateless, whereas REST is a protocol requiring stateful sessions.

REST uses standard HTTP methods, while SOAP relies on XML messaging and is protocol-based.

SOAP services are only for internal use, while REST is used for external services.

Q229. In a Spring application, a developer wants to optimize the data access layer to minimize boilerplate code. Which strategy should they follow to efficiently implement this layer?

Manually code all SQL statements in the service classes.

Extend JpaRepository and create repository interfaces for data access.

Use JDBC templates to handle all database operations.

Avoid an ORM framework and rely on raw JDBC for queries.

Q230. Which of the following annotations is used to mark a field as a foreign key?

@ForeignKey

@PrimaryKeyJoinColumn

@JoinColumn

@ForeignKeyColumn

Q231. John is developing a new feature for an application and wants to isolate his changes until they are ready for review. What should he do?

- Create a new branch
- Push changes directly to the main branch
- Delete the current branch
- Revert the last commit

Q232. What is the purpose of the @Table annotation in JPA?

- To specify the primary key of the entity.
- To define the name of the database table that the entity maps to.
- To create a one-to-many relationship between entities.
- To indicate a JSON mapping for REST APIs.

Q233. Which of the following annotations are used to define an auto-generated primary key in JPA?
A. @Entity
B. @TableGenerator
C. @GeneratedValue(strategy = GenerationType.AUTO)
D. @SequenceGenerator

- C
- A
- C and D
- B and D

Q234. Laura is developing a Java application to manage a library system. She wants to create an entity to represent a book in the database. Which annotation should she use to map her Book class to a database table?

- @Table
- @Data
- @Entity
- @Column

Q235. David is developing an application that needs to maintain a list of orders associated with a single customer. Which Spring Boot JPA annotation should he use to establish where a single entity (customer) is associated with multiple entities (orders)?

@ManyToMany

@OneToMany

@ManyToone

@OneToOne

Q236. Alex is confused about the differences between local and remote branches in git. He realizes he needs to push his changes to the shared repository. What action does he need to take?

Branch out from the current branch

Create a new local branch

Save changes locally and push to remote

Merge local branch with remote branch

Q237. A software team is implementing Git Flow for managing their development process. After implementing several features and resolving conflicts, they plan to release their product. They need to merge multiple feature branches into a release branch. What is the most effective strategy for proceeding with the merge?

Merging all feature branches into the develop branch without further review.

Testing each feature branch independently, then merging them into the release branch while ensuring everything integrates smoothly.

Merging the release branch directly back into the main branch without testing.

Deleting all feature branches and creating a new branch for release to avoid complication.

Q238. Angela is setting up a simple application and wants to store user details without using a database. What feature of Spring Security should she use?

Using JWT for token management

Using LDAP for directory access

Using InMemoryUserDetailsManager for user storage

Using JDBC for database users

Q239. What is the error in this JPA entity class that aims to use the @GeneratedValue annotation for generating primary key values?

```
1 @Entity
2 public class Product {
3     @GeneratedValue
4     private Long id;
5     private String productName;
6 }
```

The @Id annotation is missing on the id field, so it cannot be recognized as the primary key

The @GeneratedValue annotation must specify a generation strategy.

The primary key field must be of type Integer

There should not be any field of type Long in an entity

Q240. What are the main components of the Webservices architecture in java?

Service Provider, Service Requester, and Service Registry

Only Client and Service layers

Database and Server layers only

User Interface and Database layers

Q241. Linda is designing a database for a library. She needs to ensure that each book is uniquely identified and that there are no repeating groups of data in her tables. Which Normalization form should she primarily focus on?

Third Normal Form (3NF)

First Normal Form (1NF)

Second Normal Form (2NF)

Boyce-Codd Normal Form (BCNF)

Q242. In Spring, which of the following annotations is used to define a one-to-many relationship between two entities?

- A. @ManyToOne
- B. @OneToMany
- C. @OneToOne
- D. @ManyToMany

A and B

B only

C only

D only

Q243. Noah needs to create a relationship between Author and Book entities, where one author can have many books. Which annotation should Noah use on the author field in the Book entity?

@ManyToOne

@MappedSuperclass

@OneToMany

@JoinTable

Q244. Emily is setting up a new Spring Boot application. She comes across an annotation that simplifies the configuration process by enabling auto-configuration and component scanning. What is the name of this annotation?

@Configuration

@ComponentScan

@SpringBootApplication

@EnableAutoConfiguration

Q245. Lisa is designing a feature that requires ensuring multiple related database operations either all succeed or all fail. Which annotation should she use on her service method to handle these transactions properly?

@Transactional

@Service

@Autowired

@Cacheable

Q246. Emma is designing a database for a school system. She needs to ensure that each student has a unique identifier. Which feature should she implement to achieve this?

A foreign key

A primary key

A default key

An index

Q247. Alex wants to externalize his application's configuration settings in a Spring Boot project. Which file should he use to achieve this?

config.json

settings.xml

application.properties

environment.yml

Q248. What does the following JPQL query in Spring Boot return?

```
1 @Query("SELECT COUNT(o) FROM Order o WHERE o.status = :status")
2 Long countByStatus(@Param("status") String status);
3 |
```

All orders with the given status

The total number of orders matching the given status

The sum of order amounts for the given status

All unique statuses in the Order table

- Q249. What is the purpose of the @JsonIgnore annotation in Spring?
- A. To serialize a field as a JSON object
 - B. To prevent a field from being serialized or deserialized in JSON
 - C. To specify the custom format for serializing a field
 - D. To convert a JSON field to a Java object

B only

A and B

C only

D only

- Q250. David wants to set up his Spring Boot application to automatically configure itself based on the included libraries. Which annotation should he use to achieve this?

@EnableAutoConfiguration

@SpringApplication

@ComponentScan

@ConfigurationProperties

- Q251. During the implementation of a web application, Mark learns about CSRF attacks. What does Spring Security's CSRF protection aim to do?

CSRF protection prevents unauthorized commands from being executed by users.

CSRF protection creates a backup of user sessions.

CSRF protection encrypts user database connections.

CSRF protection requires validation only on login attempts.

- Q252. Tom is setting up relationships in his database for an e-commerce application. He wants to link the order table to the customer table. What term describes the key he will use in the order table?

Primary key

Composite key

Foreign key

Candidate key

Q253. In the process of handling security in her application, Emma learns about the Spring Security filter chain. What is the primary function of the security filters?

Filters process security concerns based on request and response.

Filters manage user data storage options.

Filters guarantee database connections.

Filters make web servers unnecessary.

Q254. You are normalizing the relation `Student(StudentID, Name, CourseID, InstructorID)` where `InstructorID` is determined by `CourseID`. What would be the output of executing this SQL statement aimed at reducing redundancy?

```
1 CREATE TABLE CourseInstructor AS
2 SELECT DISTINCT CourseID, InstructorID
3 FROM Student;
```

A unique list of StudentID and Name associated with InstructorID.

A unique mapping of courses to their instructors, minimizing redundancy.

A complete list of all students in every course.

An error because of missing attributes besides CourseID and InstructorID.

Q255. Mark is building a RESTful web service using Spring Boot. Which annotation is essential for returning JSON responses from his controller methods?

@GetMapping

@Controller

@RestController

@RequestMapping

Q256. What will be the output of the following SQL snippet when executed on a database with tables `Students` and `Enrollments`?

```
1 SELECT S.Name, E.Course
2 FROM Students S
3 LEFT JOIN Enrollments E ON S.StudentID = E.StudentID
4 WHERE E.Course IS NOT NULL;
```

All students and their enrolled courses, including those without enrollments

Only students who are enrolled in courses

All students, even those with NULL course values

An error due to a mismatch in selected columns

Q257. Which HTTP method is used to retrieve data from a server?

- A. POST
- B. PUT
- C. GET
- D. DELETE

A and B

C only

B and D

D only

Q258. Which of the following standards is NOT typically associated with Webservices in java?

WSDL (Web Services Description Language)

JSON (JavaScript Object Notation)

SOAP (Simple Object Access Protocol)

SQL (Structured Query Language)

Q259. What is the purpose of the @JsonBackReference annotation in Spring?

- A. To mark a field as the primary reference during serialization
- B. To prevent infinite recursion in bidirectional relationships by ignoring the back reference during serialization
- C. To serialize a field in reverse order
- D. To deserialize a field in a specific format

B only

A and B

C only

D only

- Q260. What is the purpose of the @Column annotation in JPA?
- A. To define a column in the database table that corresponds to a field in the entity class
 - B. To specify the foreign key relationship between two tables
 - C. To define the primary key column in the database
 - D. To map a database view to an entity class

A only

B only

C only

D only

- Q261. Liam is starting a new project with Spring Boot and wants to take advantage of Spring Data JPA's features. What benefits does the Spring Boot Starter Data JPA provide?

It is only for web applications

The starter provides only repository interfaces

Spring Boot Starter Data requires manual configuration

The Spring Boot Starter Data JPA simplifies dependency management.

- Q262. Which attribute of the @Column annotation is used to specify the column name in the database?

name

columnName

value

key

- Q263. While working on a team project, Lisa wants to update her local repository with the latest changes made by her coworkers that were pushed to the remote repository. Which command will she use?

git clone

git pull

git push

git fetch

Q264. A development team is collaborating on a project with multiple feature branches. Developer A has created a feature branch for a new user interface, while Developer B is working on a bug fix in a different branch. After merging their branches into the main branch, they encounter merge conflicts. What is the best approach for handling these conflicts?

By immediately deleting one of the branches to eliminate the conflict.

By reviewing the conflicting files, discussing the changes with the team, and selecting which changes to keep.

By merging the main branch back into their feature branches before merging again.

By reverting to the most recent commit of the main branch without addressing the changes from the feature branches.

Q265. Analyze the following code. What will happen if the code attempts to retrieve a user with `findById` method when no user with the specified ID exists?

```
1 Optional<User> user = userRepository.findById(100L);
2 System.out.println(user.isPresent() ? user.get().getName() : "User not found");
```

Prints "User not found"

Prints "null"

Throws a NoSuchElementException

Prints an empty string

Q266. By default, what isolation level does Spring's @Transactional use if none is explicitly specified?

READ_UNCOMMITTED

DEFAULT

SERIALIZABLE

REPEATABLE_READ

Q267. Tom has a simple database table that lists orders and their corresponding products. He realizes that each order can have multiple products and should be broken into separate tables. What is the best approach to normalize his database?

Leave the data as it is

Create separate tables for orders and products

Combine orders and products into one large table

Add an additional field to the existing table

Q268. What will be the output of the following SQL statement if the 'Sales' table contains 5 records, and all records have a column `amount` set to 100?

```
1 SELECT SUM(amount) AS total_amount
2 FROM Sales
3 WHERE amount > 50;
```

0

500

100

An error because the condition cannot be met.

Q269. In Spring Boot JPA, which type of relationship allows one entity to be associated with multiple instances of another entity?

One-to-One

Many-to-Many

One-to-Many

Self-Referencing

Q270. John is developing an application that needs to manage customer data such as names, addresses, and phone numbers. He wants to ensure that the data maintains its structure and relationships. What type of system should John use?

- A flat file system
- An in-memory data structure
- A relational database management system (RDBMS)
- A NoSQL database

Q271. Jack is working on a project that involves many different entities and wants to perform CRUD operations easily. What is a key feature of Spring Data JPA repositories that Jack should consider?

- Repositories only for read operations
- Repositories require manual implementation
- Spring Data JPA repositories simplify data access.
- Repositories are only for Hibernate

Q272. John is developing a Spring application that requires the injection of a service into a controller to handle user requests. Which method of dependency injection should he consider using for better manageability and testing?

- Using constructor-based injection.
- Using direct instantiation.
- Using method overloading.
- Using static methods.

Q273. What is the purpose of the @Query annotation in Spring Boot JPA?

- It is used to define custom SQL queries.
- It is used to define the structure of the database schema.
- It is used to specify the mapping between entities and database tables.
- It is used to define transactional boundaries.

Q274. You want to send a GET request to your Spring Boot application. Which code snippet correctly implements this?

```
1 @GetMapping("/api/products")
2 public List<Product> getAllProducts() {
3     return productService.findAll();
4 }
```

```
3 return productService.findAll(),
4 }
```

@PostMapping("/api/products") public List<Product> getAllProducts() { ... }

@RequestMapping("/api/products") public List<Product> getAllProducts() {
... }

@GetMapping("/api/products") public List<Product> getAllProducts() { ... }

@PutMapping("/api/products") public List<Product> getAllProducts() { ... }

Q275. Which component of the Spring framework is responsible for managing the lifecycle of Spring-managed beans?

Spring AOP

Spring MVC

Spring IoC

Spring Security

Q276. A development team is working on a new feature that requires parallel development efforts from multiple team members. During a merge of these branches into `main`, conflicts arise due to overlapping changes in the same file. What is the best strategy for resolving these merge conflicts effectively in Git?

Simply overwrite one branch with the other

Discard changes from both branches

Use a three-way merge to identify changes made in both branches

Commit changes without resolving conflicts

Q277. What is the purpose of the @Entity annotation in JPA?

It marks a class as a persistent entity.

It specifies the table name for a persistent entity.

It defines the primary key for a persistent entity.

It establishes a relationship between entities.

Q278. During team collaboration, Richard needs to integrate the "feature" branch into his current branch. What command should he use?

git merge feature

git checkout feature

git push feature

git branch feature

Q279. Oliver is using Spring Data JPA and needs to create a new user. Which interface should he extend to access CRUD operations easily?

JpaRepository

CrudRepository

UserRepository

UserService

Q280. Which configuration file is used for Spring XML configuration?

web.xml

application.xml

spring-context.xml

config.xml

Q281. Which is the recommended approach for configuring beans in modern Spring applications?

XML-based configuration

Java-based configuration

Annotation-based configuration

Property-based configuration

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
```

```
1 <groupId>org.springframework.boot</groupId>  
2 <artifactId>spring-boot-starter-json</artifactId>  
3 <version>2.5.4</version>  
4 </dependency>  
5  
6 |
```

Correct

Incorrect, should use spring-boot-starter-web

Incorrect, should use spring-boot-starter-data-jpa

Incorrect, should use spring-boot-starter

Q283. Which of the following code snippets demonstrates handling an HTTP POST request for creating a new record in a database using Spring Boot's JPA repository?

```
@PostMapping("/users")  
public User createUser(@Valid @RequestBody User user) {  
    return userRepository.save(user);  
}
```

```
@PostMapping("/users")  
public User createUser(User user) {  
    return userRepository.save(user);  
}
```

```
@PostMapping("/users")  
public void createUser(@Valid @RequestBody User user) {  
    userRepository(user);  
}
```

```
@PostMapping("/users")  
public void createUser(User user) {  
    userRepository.save(user);  
}
```

Q284. Which component of the Spring framework is responsible for integrating Spring applications with relational databases?

Spring AOP

Spring MVC

Spring Data

Spring ORM

Q285. How do you specify the primary bean among multiple beans of the same type?

@Autowired

@Primary

@Qualifier

@Bean

Q286. Tom is tasked with maintaining the input validation process for user registration. Which annotation can he use to ensure that a username is between 3 and 20 characters?

@Size(min=3, max=20)

@Length

@NotNull

@Range

Q287. David is working on a project and wants to see the changes he has made compared to the last commit. What command should he execute?

git diff

git status

git log

git commit

Q288. Lisa wants to track the changes in her code effectively by storing different versions of her project files. What functionality of Git should she use?

Git can only store the final version of files.

Git allows version control of all project files, including code.

Git is not suitable for binary files, only text files.

Git provides no functionality for version control.

Q289. Which of the following is NOT a core concept of Spring Framework?

Dependency Injection

Aspect-Oriented Programming

Inversion of Control

Object-Relational Mapping

Q290. Which HTTP method is used to update an existing book resource in the BookController class?

```
1 @RestController
2 @RequestMapping("/books")
3 public class BookController {
4
5     @GetMapping("/{id}")
6     public ResponseEntity<Book> getBookById(@PathVariable Long id) {
7     }
8
9     @PostMapping
10    public ResponseEntity<Book> createBook(@RequestBody Book book) {
11    }
12
13    @PutMapping("/{id}")
14    public ResponseEntity<Book> updateBook(@PathVariable Long id, @RequestBody Book book) {
15
16    }
17
18    @DeleteMapping("/{id}")
19    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
20    }
21 }
22 |
```

GET

POST

PUT

DELETE

Q291. A web application is being developed to manage customer orders. The team needs to ensure that when a new order is created, its information is accurately stored without duplication of records. What HTTP method should the team use to create the order when making requests to the REST API?

POST is used to create new resources.

GET is used to create new resources.

DELETE is used to remove existing resources.

PUT is used to update an existing resource.

Q292. Which Spring component is used for building web applications and RESTful APIs?

Spring AOP

- Spring MVC
- Spring IoC
- Spring Security

Q293. Which of the following annotations is used in JPA to specify a primary key column in an entity?

- @PrimaryKey
- @Id
- @Column
- @GeneratedValue

Q294. When using @Autowired annotation for dependency injection, which of the following options is used to resolve ambiguity when multiple beans of the same type are available?

- @Qualifier
- @Primary
- @Value
- @Resource

Q295. In Spring Boot, which annotation is commonly used to mark a class as a JPA entity?

- @Entity
- @Table
- @Persistent
- @Data

Q296. What is the base URL for accessing the book-related endpoints in this Spring Boot REST API?

```
1 @RestController
2 @RequestMapping("/api/books")
3 public class BookController {
4
5     @Autowired
6     private BookService bookService;
7
8     @GetMapping("/{id}")
9     public ResponseEntity<Book> getBookById(@PathVariable("id") Long id) {
10         Book book = bookService.getBookById(id);
11         if (book != null) {
12             return ResponseEntity.ok(book);
13         }
14     }
15 }
```

```
13     } else {
14         return ResponseEntity.notFound().build();
15     }
16 }
17
18 @PostMapping
19 public ResponseEntity<Book> createBook(@RequestBody Book book) {
20     Book createdBook = bookService.createBook(book);
21     return ResponseEntity.status(HttpStatus.CREATED).body(createdBook);
22 }
23 }
```

- /api/books
- /books
- /api
- /api/book

Q297. You are working on a Java project and need to manage dependencies automatically. Which tool can help you manage your project’s build process and dependencies?

- Gradle
- Ant
- Maven
- JUnit

Q298. Jessica has just finished working on a feature and wants to check which changes are staged for the next commit. Which command should she use?

- git diff
- git status
- git log
- git fetch

Q299. In a customer feedback system, you need to expose a REST API endpoint to accept feedback. Which annotation in Spring Boot is used to handle HTTP POST requests?

- @GetMapping
- @PostMapping

@PutMapping

@RequestBody

Q300. Which of the following methods is commonly used in Spring Boot JPA repositories to retrieve all entities of a specific type?

findAll()

findById()

save()

delete()

Q301. In a hotel booking system, which HTTP method is used when a user submits a form to book a room?

GET

POST

PUT

DELETE

Q302. To test a new feature, Daniel creates a branch called `feature-x`. What command does he use to switch to this newly created branch?

git move feature-x

git checkout feature-x

git create feature-x

git go feature-x

Q303. Emma is implementing a service for retrieving user settings stored as key-value pairs. Which structure in Spring would best fit her use case?

List

Set

Queue

Map

Q304. You want to implement a method in a Spring Boot application to handle a POST request that creates a new product. What is the correct code snippet?

```
1 @PostMapping("/api/products")
2 public ResponseEntity<Product> createProduct(@RequestBody Product product) {
3     Product createdProduct = productService.save(product);
4     return new ResponseEntity<>(createdProduct, HttpStatus.CREATED);
5 }
```



```
@PutMapping("/api/products") public ResponseEntity<Product>
createProduct(@RequestBody Product product) { ... }
```

```
@PostMapping("/api/products") public void createProduct(@RequestBody
Product product) { ... }
```

```
@PostMapping("/api/products") public ResponseEntity<Product>
createProduct(@RequestBody Product product) { ... }
```

```
@RequestMapping(value="/api/products", method=RequestMethod.POST)
public ResponseEntity<Product> createProduct(@RequestBody Product
product) { ... }
```

Q305. Which annotation is used to mark a class as a Spring bean in annotation-based configuration?

@Service

@Bean

@Component

@Autowired

Q306. How is the user ID passed to the updateUser method?

```
1 @RestController
2 @RequestMapping("/users")
3 public class UserController {
4
5     @GetMapping("/{id}")
6     public ResponseEntity<User> getUserById(@PathVariable Long id) {
7     }
8
9     @PostMapping("/")
10    public ResponseEntity<User> createUser(@RequestBody User user) {
11    }
12
13    @PutMapping("/{id}")
14    public ResponseEntity<User> updateUser(@PathVariable Long id, @RequestBody User user) {
15    }
16
17    @DeleteMapping("/{id}")
18    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
19    }
20 }
```



As a query parameter

As a request body

As a path variable

As a request header

Q307. What is a servlet in Java?

A client-side scripting language

A server-side program that extends the functionality of a web server

A markup language used for web page structuring

A database management system

Q308. In Spring Boot, how do you customize the JSON serialization behaviour globally?

By configuring Jackson2ObjectMapperBuilder bean

By setting properties in application.properties

By using @JsonSerialize on classes

By creating a custom JSON serializer class

Q309. Which of the following statements is true about constructor injection in Spring?

Constructor injection is not supported in Spring.

Constructor injection can only be used for primitive types.

Constructor injection is preferred over setter injection for mandatory dependencies.

Constructor injection can only be used with XML-based configuration.

Q310. What is the output of the following code snippet that uses Spring Boot's @Autowired annotation to inject a bean into another bean?

```
1 @Component
2 public class MyBean {
3     private MyOtherBean otherBean;
4
5     @Autowired
```

```
5  @Component
6  public MyBean(MyOtherBean otherBean) {
7      this.otherBean = otherBean;
8  }
9  // bean logic
10 }
11
12 @Component
13 public class MyOtherBean {
14     // other bean logic
15 }
```

The code does not compile because MyOtherBean is not annotated with @Component.

The code does not compile because the constructor of MyBean is not public.

The code runs without errors and successfully injects the MyOtherBean bean into the MyBean bean.

Complier error.

Q311. Which HTTP response code indicates that the requested resource is temporarily unavailable, but may be available in the future?

200

301

503

403

Q312. Maven is built on which programming language?

Java

Python

C++

Ruby

Q313. In a multi-module Maven project for an e-commerce system, you want to ensure all modules are built in the correct order. Which section of the pom.xml helps define the dependencies between modules?

<dependencies>

<build>

<modules>

<properties>

Q314. When merging his branch into the main branch, Omar sees a message indicating a conflict. What does this mean?

His branch will be deleted automatically.

There is conflicting code between the two branches that needs resolution.

The merge will occur without any issues.

Git will send a notification to the team about the conflict.

Q315. Jacob's application needs to validate that a user must have at least one role. Which annotation should he use to enforce this rule in his Spring Boot application?

@Size(min=1)

@Max

@NotEmpty

@Required

Q316. In a cake shop application, you want to inject a service class into your controller. What is the recommended annotation to define a Spring service class?

@Service

@Component

@Autowired

@Bean

Q317. A user requests a specific page from your travel website. Which method in a servlet handles this GET request?

doPost()

doGet()

doService()

doDispatch()

Q318. What will be the URL endpoint to create a new book using the below controller?

```
1 @RestController
2 @RequestMapping("/api/books")
3 public class BookController {
4     @GetMapping
5     public List<Book> getAllBooks() {
6     }
7
8     @PostMapping
9     public ResponseEntity<Book> createBook(@RequestBody Book book) {
10    }
11    @GetMapping("/{id}")
12    public ResponseEntity<Book> getBookById(@PathVariable("id") Long id) {
13    }
14    @PutMapping("/{id}")
15    public ResponseEntity<Book> updateBook(@PathVariable("id") Long id, @RequestBody Book book) {
16    }
17    @DeleteMapping("/{id}")
18    public ResponseEntity<Void> deleteBook(@PathVariable("id") Long id) {
19    }
20 }
```

POST /api/books

GET /api/books

PUT /api/books

DELETE /api/books

Q319. Charlie is setting up Git for his development environment. What is the purpose of using SHA1 (Secure Hash Function) in Git?

To encrypt files

To track changes in files

To name and identify objects

To manage branching and merging

Q320. In JPA, what is the purpose of the EntityManager interface?

To define persistence operations and manage entity lifecycle.

To execute SQL queries and interact with the database.

To provide an interface for configuring the JPA implementation.

To manage transactions and ensure data integrity.

Q321. What is the purpose of the JpaRepository interface in Spring Boot JPA?

- To define basic CRUD (Create, Read, Update, Delete) operations for an entity.
- To execute native SQL queries and interact with the database.
- To define custom queries using JPQL (Java Persistence Query Language).
- To manage transactions and ensure data integrity.

Q322. Sarah’s application throws an `IllegalArgumentException` when she passes a null value to a method expecting a user ID. Which annotation can she use to help prevent this in future requests?

- @NotBlank
- @Valid
- @ValidRequest
- @NotNull

Q323. What is the purpose of the @Autowired annotation in Spring?

- To indicate a dependency that should be automatically injected
- To define a new bean in the Spring application context
- To enable Spring MVC support
- To handle exceptions thrown by Spring beans

Q324. You want to customize the build lifecycle of your project to run tests before packaging the application. In which section of the pom.xml would you configure the order of plugin executions?

- <dependencies>
- <properties>
- <build>
- <profiles>

Q325. What will be the URL pattern to access the getUserById endpoint for a user with ID 123?

```
1 @RestController
2 @RequestMapping("/api")
3 public class UserController {
4
```

```

5  @GetMapping("/users")
6  public List<User> getUsers() {
7
8  }
9
10 @GetMapping("/users/{id}")
11 public ResponseEntity<User> getUserById(@PathVariable Long id) {
12
13 }
14 }|

```

/api/users

/api/users/{id}

/api/users/123

/api/users?id=123

Q326. You need to handle an HTTP request and return a JSON response from your Spring Boot application. Which code snippet correctly implements this?

```

1  @GetMapping("/api/user/{id}")
2  public ResponseEntity<User> getUserById(@PathVariable Long id) {
3      User user = userService.findById(id);
4      return ResponseEntity.ok(user);
5  }|

```

@GetMapping("/api/user/{id}") public ResponseEntity<User>
getUserById(@PathVariable Long id) { ... }

@PostMapping("/api/user/{id}") public ResponseEntity<User>
getUserById(@PathVariable Long id) { ... }

@RequestMapping("/api/user/{id}") public ResponseEntity<User>
getUserById(@PathVariable Long id) { ... }

@PutMapping("/api/user/{id}") public ResponseEntity<User>
getUserById(@PathVariable Long id) { ... }

Q327. Which HTTP method should be used to retrieve a specific book by its ID in this REST API?

```

1  @RestController
2  @RequestMapping("/api/books")
3  public class BookController {
4
5      @Autowired
6      private BookService bookService;
7
8      @GetMapping("/{id}")
9      public ResponseEntity<Book> getBookById(@PathVariable("id") Long id) {
10         Book book = bookService.getBookById(id);
11         if (book != null) {
12             return ResponseEntity.ok(book);
13         } else {
14             return ResponseEntity.notFound().build();
15         }
16     }
17
18     @PostMapping

```

```
19 public ResponseEntity<Book> createBook(@RequestBody Book book) {
20     Book createdBook = bookService.createBook(book);
21     return ResponseEntity.status(HttpStatus.CREATED).body(createdBook);
22 }
23 }
```

GET

POST

DELETE

PUT

Q328. John is developing a Spring application where he needs to manage the lifecycle of his beans. He is currently using the default scope for most of his beans. However, he realizes he doesn't fully understand what the default scope entails. What is the default scope of a Spring bean in John's application?

Prototype

Singleton

Request

Session

Q329. You need to automatically inject dependencies in your Spring application. Which annotation should you use in your controller?

@Inject

@Autowired

@Resource

@Qualifier

Q330. Mark receives an `EntityNotFoundException` while trying to fetch user data. What does this exception indicate in his Spring application?

The database is currently down.

The requested entity does not exist in the database.

There are connectivity issues with the database.

The query syntax is incorrect.

Q331. You are working on a restaurant website and want to include a form where users can make reservations. How would you ensure the form data is sent to the server using POST method?

<form method="get">

<form action="post">

<form method="post">

<form type="post">

Q332. Which HTTP status code indicates a successful request?

200

404

500

302

Q333. Which HTTP method is typically used to retrieve data from a server using a servlet?

GET

POST

PUT

DELETE

Q334. What is the purpose of the @Qualifier annotation in Spring?

To identify primary beans

To specify a bean name

To define bean scopes

To resolve bean ambiguities

Q335. What is the role of the DispatcherServlet in a Spring MVC application?

It handles all HTTP requests and responses.

It manages database connections.

It defines mappings between URL paths and controllers.

It processes asynchronous tasks.

Q336. What will happen if the findById method of the UserRepository in the getUserById method returns an empty Optional?

```
1 @Service
2 public class UserService {
3
4     @Autowired
5     private UserRepository userRepository;
6
7     public User getUserById(String userId) {
8         return userRepository.findById(userId)
9             .orElseThrow(() -> new UserNotFoundException("User not found with ID: " + userId));
10    }
11
12    public User createUser(User user) {
13        // Validation and save logic
14        return userRepository.save(user);
15    }
16 }
```

It will return null as the result.

It will throw an exception of type UserNotFoundException.

It will return an empty User object.

It will return a default User object.

Q337. What is the expected HTTP method and URI to retrieve a list of users from this API?

```
1 @RestController
2 @RequestMapping("/api")
3 public class UserController {
4
5     @GetMapping("/users")
6     public List<User> getUsers() {
7         List<User> users = new ArrayList<>();
8         return users;
9     }
10
11    @PostMapping("/users")
12    public User createUser(@RequestBody User user) {
13        return user;
14    }
15 }
```

GET /api/users

POST /api/users

GET /users

POST /users

Q338. What is the purpose of the EntityManager interface in Spring Boot JPA?

To define persistence operations and manage entity lifecycle.

To execute SQL queries and interact with the database.

To manage transactions and ensure data integrity.

To provide an interface for configuring the JPA implementation.

Q339. Which of the following is NOT a scope in Spring?

Singleton

Prototype

Request

Static

Q340. John wants to extract a list of user names from a database and maintain their order. What collection would be most appropriate for this task in Spring Boot?

Set

List

Queue

Map

Q341. What is the output of the following code snippet that uses Spring Boot's @Transactional annotation to wrap a method call in a transaction?

```
1 @Service
2 public class MyService {
3
4     @Transactional
5     public void updateSomething() {
6         // update something in the database
7     }
8 }
9
10 @SpringBootApplication
11 public class MyApp {
12
13     public static void main(String[] args) {
14         ConfigurableApplicationContext context = SpringApplication.run(MyApp.class, args);
15         MyService myService = context.getBean(MyService.class);
16         myService.updateSomething();
17     }
18 }
19 |
```

The code does not compile because the @Transactional annotation is not used correctly.

The code runs without errors but does not wrap the updateSomething() method in a transaction.

The code runs without errors and wraps the updateSomething() method in a transaction.

Compiler error.

Q342. Which annotation is commonly used in the Spring Framework to mark a class as a Spring bean?

@Component

@Service

@Repository

All of the above

Q343. A developer is creating a RESTful service in Spring Boot that requires the use of JPA repositories for CRUD operations. The service needs to simplify database interactions without writing repetitive code for each operation. What is the key advantage of using Spring Data JPA repositories in this context?

They eliminate the need for writing boilerplate code for common data access operations.

They require developers to write complex SQL queries for each CRUD operation.

They are only useful for interactions with NoSQL databases.

They mandate the use of specific data sources for persistence.

Q344. What is the primary benefit of using JPA over Hibernate directly?

JPA provides better performance and efficiency in database operations.

JPA provides a standard API, allowing for vendor-independent code.

JPA offers more advanced features and functionalities than Hibernate.

JPA simplifies the mapping process and reduces boilerplate code.

Q345. Which of the following statements is true about Hibernate and JPA?

Hibernate is a specific implementation of JPA.

JPA is a specific implementation of Hibernate.

Hibernate and JPA are unrelated and serve different purposes.

Hibernate and JPA can be used interchangeably without any differences.

Q346. While debugging an application, Sophia discovers a `DataIntegrityViolationException`. What is the common reason for this error in a Spring Boot application?

The database connection string is incorrect.

Data being inserted violates constraints set in the database schema.

There is a syntax error in the query.

The method is called without the required inputs.

Q347. What will be the HTTP method and endpoint for creating a new user in the UserController?

```
1 @RestController
2 public class UserController {
3
4     @Autowired
5     private UserService userService;
6
7     @GetMapping("/users/{id}")
8     public ResponseEntity<User> getUserById(@PathVariable("id") String userId) {
9         User user = userService.getUserById(userId);
10        return ResponseEntity.ok(user);
11    }
12
13    @PostMapping("/users")
14    public ResponseEntity<User> createUser(@RequestBody User user) {
15        User createdUser = userService.createUser(user);
16        return ResponseEntity.status(HttpStatus.CREATED).body(createdUser);
17    }
18 }
```

GET /users/{id}

POST /users

PUT /users/{id}

DELETE /users/{id}

Q348. You want to create a new Spring Boot application with JPA support. Which of the following snippets correctly configures the DataSource?

```
1 @Bean
2 public DataSource dataSource() {
3     return DataSourceBuilder.create()
4         .driverClassName("com.mysql.cj.jdbc.Driver")
5         .url("jdbc:mysql://localhost:3306/mydb")
6         .username("user")
7         .password("password")
```

```
8     .build();
9 }|
```

@Configuration public DataSource dataSource() { ... }

@Bean public DataSource dataSource() { ... }

@Autowired public DataSource dataSource() { ... }

@Service public DataSource dataSource() { ... }

Q349. Which Spring component provides authentication and authorization capabilities for securing Spring-based applications?

Spring AOP

Spring MVC

Spring IoC

Spring Security

Q350. Which of the following statements about Spring's IoC container is true?

The IoC container creates and manages singleton beans only.

The IoC container can create and manage beans with different scopes, including singleton and prototype

The IoC container reuses prototype bean instances after the first creation

The IoC container is responsible for injecting dependencies into beans using the @Inject annotation

Q351. What will be the HTTP method used to access the createUser endpoint?

```
1 @RestController
2 @RequestMapping("/api")
3 public class UserController {
4     @GetMapping("/users")
5     public List<User> getUsers() {
6     }
7     @PostMapping("/users")
8     public ResponseEntity<String> createUser(@RequestBody User user) {
9
10    }
11 }|
```

GET

POST

PUT

DELETE

Q352. Which of the following modules of the Spring Framework is responsible for dependency injection and inversion of control (IoC)?

Spring MVC

Spring Core

Spring Data

Spring Boot

Q353. What is the purpose of the Spring Data module?

To handle transactions

To provide support for JDBC

To simplify database access

To handle messaging

Q354. In Spring Boot JPA, which annotation is used to define a primary key field in an entity class?

@PrimaryKey

@Id

@Column

@GeneratedValue

Q355. Alice wanted to use IoC containers to assemble the dependencies between the objects. Which of the following are correct IoC containers that she can use in Spring?

- BeanFactory, ApplicationContext, locContextFactory
- BeanFactory, BeanContext, locContextFactory
- BeanFactory, ApplicationContext
- BeanFactory, ApplicationContext, BeanContext

Q356. What does the HTTP response code 404 indicate?

- Unauthorized access
- Successful request with no content
- Resource not found
- Internal server error

Q357. You are using Spring Data JPA and want to create a custom query to find users by their email. What is the correct repository method?

- List<User> findByEmail(String email);
- @Query("SELECT u FROM User u WHERE u.email = ") List<User> findByEmail(@Param("email") String email);
- @Query("SELECT * FROM User WHERE email = ?1") List<User> findByEmail(String email);
- @Query("SELECT u FROM User u WHERE u.email = ?1") User findByEmail(String email);

Q358. A Spring Data JPA repository is designed to fetch a paginated list of `Product` objects from the database. The developer has identified the need to sort these products by price in descending order while paginating the results. How should the developer implement this?

- By using the Pageable interface with Sort.Order(price, Sort.Direction.DESC) in the repository method.
- By manually implementing pagination and sorting in the service layer without leveraging JPA features.
- By adding sorting parameters directly in the SQL query string.

By creating a custom pagination service that handles sorting independently.

Q359. What will be the HTTP response status code after successfully creating a new product?

```
1 @RestController
2 @RequestMapping("/api")
3 public class ProductController {
4
5     @PostMapping("/products")
6     public ResponseEntity<Void> createProduct(@RequestBody Product product) {
7         // Save product to the database
8         return ResponseEntity.created(URI.create("/api/products/" + product.getId())).build();
9     }
10 }
```

200

201

204

400

Q360. In a Spring Boot application, a developer is trying to establish a one-to-many relationship between a `Customer` entity and an associated `Order` entity. Which of the following configurations correctly implements this relationship?

Use @ManyToOne annotation on the Order entity and @OneToMany on the Customer entity.

Apply @OneToOne annotation on both entities.

Use @OneToMany annotation on the Order entity and @ManyToOne on the Customer entity.

Apply @ManyToOne on Customer and @OneToOne on Order entity.

Q361. You are building a microservice that fetches data from a database. How do you inject the service that interacts with the database into a Spring Boot controller?

@Service

@Repository

@Autowired

@Component

Q362. What is the purpose of the @RequestBody annotation in a Spring Boot controller method?

It represents the request parameters passed in the URL.

It defines the URL mapping for the controller method.

It specifies the HTTP method to be used for the request.

It indicates that the method handles a request with a JSON payload.

Q363. Which protocol is commonly used for communication between a web client and a web server?

HTTP

FTP

SMTP

DHCP

Q364. What is the purpose of the ApplicationContext interface in Spring's IoC container?

It represents the configuration metadata that the container uses to create and manage beans.

It serves as the main interface for accessing and interacting with the Spring IoC container.

It provides a mechanism for performing aspect-oriented programming (AOP) in Spring applications.

It defines a set of standard annotations for dependency injection in Spring.

Q365. Grace's API returns a list of products each time a user accesses the products endpoint. What Spring annotation should she use to map the HTTP GET request to this method?

@PutMapping

@PostMapping

@GetMapping

@RequestMapping

Q366. Michelle wants to ensure a specific method in her service class runs within a transaction, regardless of how many times it is called. What annotation should she apply?

@Service

@Transactional

@Configuration

@Autowired

Q367. In client-server communications, what action does the client perform to initiate a request?

The client waits for a response from the server.

The client sends a request to the server.

The client updates the server data automatically.

The client listens for data from other clients.

Q368. What are Webservices primarily designed for in the context of networked applications?

To store large amounts of data efficiently

To enable interoperable machine-to-machine communication

To create graphical user interfaces

To enhance the performance of operating systems

Q369. Alex is working with XML configuration in a Spring project and needs to add a bean for a `UserService`. Which element must he use in his XML configuration file to define this bean properly?

element

<bean>

class

component

Q370. Lila wants to set up her Spring application such that it automatically resolves the dependencies for her `PaymentService` based on the types of the injected components rather than the names. Which autowiring mode should she use to achieve this?

Autowiring by Constructor

Autowiring by Name

Autowiring by Type

No Autowiring

- Q371. Emily is developing a Spring application where she needs to ensure that her service class requires a repository at construction time. She wants to enforce immutability for the repository reference within the service. Which type of dependency injection should she use?

Setter Injection

Field Injection

Interface Injection

Constructor Injection

- Q372. A developer is tasked with implementing a service that requires a mandatory database connection at the time of object creation. The developer is considering using Dependency Injection in Spring. Which method of Dependency Injection would be the most appropriate choice to ensure that the database connection is always provided when the service is instantiated?

Setter Injection, allowing the database connection to be set post-instantiation.

Constructor Injection, ensuring the database connection is passed at the time of object creation.

Field Injection, where the database connection is automatically set during the bean initialization.

Method Injection, invoking a method to supply the database connection after the object is created.

- Q373. Alex is developing a web application and wants to ensure that different components can work together without being tightly coupled. Which feature of the Spring framework should he use to effectively manage dependencies between these components?

Annotations

Configuration files

Dependency Injection

Direct object instantiation

- Q374. Emma is building a Spring Boot application and wants to minimize the amount of configuration she has to write. Which feature of Spring Boot will help her automatically configure components without needing extensive manual configurations?

Manual bean definition

Spring Initializr

Auto-configuration

XML configuration

Q375. Linda is trying to send a new book entry to her Spring Boot application using Postman. What steps must she follow to correctly perform a POST request?

Choose the "GET" method, set the URL, and include the book's details in the URL parameters.

Select the "POST" method, ensure the URL points to the correct endpoint, and include the book's JSON representation in the body.

Use the "PUT" method, provide the URL, and add the book's details in the headers.

Select "DELETE" method, provide the URL, and send the book's details as a query string.

Q376. John is developing a RESTful API for a bookstore. He wants to allow users to retrieve the list of all available books. Which HTTP method should he use in his Spring Boot application to implement this functionality?

POST

DELETE

PUT

GET

Q377. Moving forward with her API development, Mia wants to return JSON data after processing a GET request in Spring Boot. Which annotation should she use to ensure that her response is sent as JSON?

@ResponseBody

@RequestMapping

@Primary

@GetMapping

Q378. Tom is developing a Spring Boot REST API that processes user login requests. When a user successfully logs in, which HTTP response code should Tom return to indicate that the login request was successful?

- 404 Not Found
- 200 OK
- 400 Bad Request
- 500 Internal Server Error

Q379. During testing, David encounters a scenario where his Spring Boot REST API throws an exception due to a failure in the database connection. Which HTTP response code would be most appropriate for David to return in this situation to indicate an internal server error?

- 401 Unauthorized
- 200 OK
- 404 Not Found
- 500 Internal Server Error

Q380. Lisa is building an e-commerce application. One of the endpoints is for retrieving product details. If a request is made for a product that does not exist in the database, which HTTP response code should she return to inform the user that the requested product was not found?

- 200 OK
- 403 Forbidden
- 500 Internal Server Error
- 404 Not Found

Q381. A developer is working on a Spring Boot application and needs to ensure that the service layer effectively interacts with the repository layer while maintaining the business logic. If they introduce a feature that retrieves users by their unique ID but find that they sometimes receive a 404 response, what does this indicate in the context of their REST API?

- The server is experiencing internal issues preventing it from fulfilling the request.
- The requested user ID does not exist in the database.
- The API call was made without sufficient authentication.
- The server successfully processed the request, but the user data is not returned due to an API filter.

Q382.

Imagine a scenario where a team is testing a Spring Boot REST API for an e-commerce application. They encounter a situation where the API returns a 500 response after attempting to access a specific product by ID. What could be the most likely reason for this response?

- The product ID submitted was invalid or out of range.
- The resource was found, but there was an issue with the database operation.
- The server has been deployed with incorrect environment variables.
- The request format was incorrect, leading to failure in processing.

Q383. David is tasked with creating a custom exception in his Spring Boot service to indicate when a requested resource is not found. Which statement describes how he should define this custom exception?

- A custom exception must implement the Serializable interface.
- A custom exception class extends the Exception class to define specific error conditions.
- A custom exception class is created using an interface.
- A custom exception class must be annotated with `@RestController`.

Q384. Emma is developing a Spring Boot application that frequently encounters errors, such as data not being found. She wants to ensure that her application handles exceptions gracefully. What will happen if an exception is thrown and no specific handling is implemented?

- The application crashes without any response.
- The application automatically retries the request.
- Spring Boot provides a default error response to the client.
- The exception is logged but not propagated to the client.

Q385. James is working on a banking application where he needs to implement exception handling to deal with cases when users attempt to withdraw more money than they have in their account. He considers creating a custom exception named ``InsufficientFundsException``. What should he ideally include in this custom exception class to provide meaningful feedback when the exception is thrown?

- A constructor that only takes a String message to indicate the reason for the exception.
- A method that logs the exception details to the database whenever it is thrown.

Additional fields to store account details and the required amount to make the exception context richer.

An empty constructor so that the exception can be instantiated without any data.

Q386. Emma is building a user management system in her Springboot application. She needs to ensure that each user can only be registered once, based on their usernames. What is the most suitable Collection type for storing the usernames?

ArrayList

HashMap

Set

LinkedList

Q387. A developer is working on a library management system where each book has a unique ISBN number and several attributes like title, author, and genre. They need a solution to store and manage these books efficiently, allowing for quick lookup by ISBN. Which Springboot application for CRUD operations using Collections should the developer use to best suit these requirements?

List of Books

HashMap with ISBN as the key

Array of Books

TreeSet of Books

Q388. John is developing a new application that requires him to manage the relationship between users and the products they purchase. He wants to avoid writing complex SQL joins and code to manually handle these relationships. Which advantage of using ORM over JDBC would best suit John's needs in this scenario?

ORM provides automatic mapping of objects to database tables, reducing manual coding effort.

JDBC requires complex SQL queries to map objects manually.

ORM does not support relationships between objects.

JDBC automatically maps objects to tables without extra work.

Q389. Lisa is maintaining an application that requires her to track and manage transactions with minimal coding overhead. She notices that her team has been writing repetitive code to handle transactions using JDBC. Which feature of ORM would help simplify transaction management for Lisa's team?

ORM requires more code to handle transactions than JDBC.

Transactions in ORM can only be managed manually without framework support.

ORM simplifies transaction management through built-in methods, reducing boilerplate code.

JDBC automatically handles rollback for all operations without user input.

- Q390. Michael is tasked with migrating a legacy system that uses JDBC to a modern application stack. One of his key requirements is to reduce the amount of boilerplate code needed for database operations. He considers adopting ORM for this migration. What is the main advantage Michael should highlight when discussing this shift to his team?

ORM can require extensive manual configuration compared to JDBC.

ORM abstracts away SQL complexities, allowing developers to focus on business logic.

ORM lacks support for batch operations, which JDBC handles efficiently.

ORM's performance is inherently slower due to data abstraction.

- Q391. Tom is developing a new Java application that requires him to create a class that represents a user in a database. To ensure this class is recognized as an entity by Hibernate, which annotation should he use?

@Table

@Entity

@Column

@Id

- Q392. Lisa is tasked with creating a new application that requires tracking multiple orders associated with a single customer. Which annotation should she use in her Customer entity class to demonstrate this one-to-many relationship?

@ManyToOne

@OneToMany

@JoinColumn

@Column

- Q393. James is implementing a database schema in his application and needs to specify which attribute serves as the unique identifier for his Product entity. Which annotation will he need to apply to the attribute representing the product ID?

- @GeneratedValue
- @Id
- @Version
- @Transient

Q394. A developer is working on a library management system and needs to create a class to represent a book entity in the database. The class should allow Hibernate to know that this is a persistent entity, and it should be mapped to a table named "books" with an auto-generated ID for each book. What annotations should the developer use to achieve this?

- @Transient and @Entity
- @Table(name="books") and @Id
- @Entity and @GeneratedValue
- @Entity, @Table(name="books"), and @Id and @GeneratedValue

Q395. John is developing a Spring Boot application and needs to define a class that corresponds to a database table for storing customer data. Which JPA annotation should he use at the top of his class to indicate that it is a JPA entity?

- @Table
- @Column
- @Entity
- @Id

Q396. Chris needs to configure an auto-incrementing field for the order ID in his entity class. Which JPA annotation is appropriate for this purpose?

- @Id
- @Table
- @Column
- @GeneratedValue

Q397. Rachel wants to map a Java class field to a specific column in the database and requires that the field name does not match the column name. Which JPA annotation should she use on her field to manage this mapping?

@Entity

@Table

@Id

@Column

Q398. Emily is working on a project where she needs to specify a unique identifier for a product in her database. Which annotation should she apply to the field in her entity class to designate it as the primary key?

@GeneratedValue

@Entity

@Id

@Column

Q399. Sarah is tasked with creating a repository for an application that manages books. She needs to retrieve a list of books by a specific author's last name. Which method naming convention should she follow to ensure that Spring Data JPA can automatically implement this function?

listByAuthorLastName

getBooksByAuthorLastName

findBooksByLastName

retrieveBooksByAuthor

Q400. A team is developing an e-commerce application where you need to model the relationship between customers and orders. Each customer can place multiple orders, but each order belongs to a single customer. Which Spring JPA annotation should be used in the Customer entity to represent this one-to-many relationship correctly?

@OneToOne

@OneToMany

@ManyToMany

@Embedded

Q401. Maria is configuring JPA relationships in her application and needs to define how entities should fetch related data. She prefers to load the related entity only when it is explicitly accessed to optimize performance. Which fetch type should she use for this purpose?

EAGER

LAZY

JOIN FETCH

FETCH

- Q402. Mark is implementing a transaction management system in a banking application using Spring Boot JPA. He needs to ensure that a series of operations related to transferring money from one account to another either all succeed or fail together. Which annotation will he need to apply to the service method to achieve this?

@Rollback

@Transactional

@Async

@Scheduled

- Q403. Jessica is developing a Springboot application that needs to select all the products available in the database. Which JPQL query should she use to achieve this?

SELECT * FROM Product

SELECT p FROM Product p

SELECT p.name FROM Product

SELECT count(p) FROM Product p

- Q404. Tom wants to calculate the average price of all books in a library database using JPQL in a Springboot application. Which query would be most appropriate for him to use?

SELECT AVG(b.price) FROM Book b

SELECT SUM(b.price) FROM Book b

SELECT b.price FROM Book b

SELECT COUNT(b) FROM Book b

- Q405. Emily has a requirement to retrieve users from the database who are older than 30 years using a JPQL query. Which JPQL clause should she incorporate to filter her results appropriately?

```
SELECT u FROM User u WHERE u.age > 30
```

```
SELECT u FROM User u ORDER BY u.age
```

```
SELECT u FROM User u GROUP BY u.age
```

```
SELECT u FROM User u HAVING u.age > 30
```

- Q406. Alex is working on a Springboot application that requires him to join the `Order` and `Customer` entities to fetch order details along with customer information. Which query structure should Alex use to accomplish this?

```
SELECT o FROM Order o JOIN Customer c ON o.customerId = c.id
```

```
SELECT o, c FROM Order o JOIN o.customer c
```

```
SELECT o, c FROM Order o, Customer c
```

```
SELECT o FROM Order o INNER JOIN Customer c
```

- Q407. A developer needs to implement a feature that retrieves the total sales amount from a `Sales` entity grouped by product category in a Spring Boot application using JPQL. Which of the following queries would achieve this?

```
SELECT s.category, SUM(s.amount) FROM Sales s GROUP BY s.category
```

```
SELECT SUM(s.amount) FROM Sales s ORDER BY s.category
```

```
SELECT s.category, AVERAGE(s.amount) FROM Sales s GROUP BY  
s.category
```

```
SELECT DISTINCT s.category, SUM(s.totalAmount) FROM Sales s
```

- Q408. In a Spring Boot application, a developer is required to retrieve a count of all orders placed by a specific customer using JPQL. The entity model includes `Customer` and `Order`, where a customer can have multiple orders. What would be the correct JPQL query for this requirement?

```
SELECT o FROM Order o WHERE o.customer.id = :customerId
```

```
COUNT(SELECT o FROM Order o WHERE o.customerId = :customerId)
```

```
SELECT COUNT(o) FROM Order o WHERE o.customer.id = :customerId
```

```
COUNT(o) FROM Orders o WHERE o.customerId = :customerId
```

- Q409. A junior developer is trying to grasp the concept of joins in JPQL while fetching user data along with their associated orders. Which of the following JPQL queries correctly implements an inner join between `User` and `Order` entities based on a foreign key relationship where the `Order` entity has a field `user` that refers to the `User` entity?

```
SELECT u FROM User u INNER JOIN u.orders o
```

```
SELECT u, o FROM User u, Order o WHERE u.id = o.userId
```

```
SELECT u FROM User u JOIN Order o ON u.id = o.userId
```

```
JOIN User u WITH Order o
```

- Q410. A developer is tasked with implementing a feature where users can update their profile information. The user entity has attributes like name, email, and age. The developer needs to write a JPQL update query to modify the user's email based on their ID. Which of the following JPQL queries correctly accomplishes this task?

```
UPDATE User u SET u.email = :email WHERE u.id = :id
```

```
MODIFY User SET email = :email WHERE id = :id
```

```
CHANGE User u SET u.email TO :email FOR u.id = :id
```

```
UPDATE Users SET email = :email WHERE id = :id
```

- Q411. Emily is designing a new database for her online bookstore. She realizes that she needs to ensure that the same book cannot be listed multiple times in her database. What is the importance of normalization in her database design?

Normalization makes the database slower by increasing redundancy.

Normalization is only a step during the backup process.

Normalization helps eliminate duplicate data, ensuring efficient storage and maintenance.

Normalization is irrelevant for small datasets.

- Q412. John is working on a database for his company's customer management system. He knows he needs to format the customer data to avoid having multiple phone numbers in the same column. Which level of normalization should he achieve to ensure that all entries in a column are atomic and unique?

2NF

3NF

1NF

Denormalized

- Q413. Lucy's team is analyzing the sales data from their e-commerce platform. She notices that the same product details are listed multiple times across different entries. This data arrangement might slow down their reporting process. Which scenario best describes this issue?

Lucy's data is denormalized, leading to multiple entries of the same product.

Lucy has ensured that all product information is stored in separate tables.

Lucy's database is in 1NF with no repeating groups or ungrouped attributes.

Lucy's data is organized in a normalized fashion preventing redundancy.

Q414. John is developing an application where each user has one unique profile. How should he define the relationship between the User entity and the Profile entity in Spring Boot JPA?

Use the @ManyToOne annotation for the User and Profile relationship.

Use the @OneToOne annotation between the User and Profile entities.

Implement a @OneToMany relationship for this scenario.

Abandon the relationship as it cannot be established in JPA.

Q415. Emily's task is to manage the personal information of customers, where each customer has exactly one address. What operation should she implement to retrieve a customer's address based on their ID in Spring Boot JPA?

Use the DELETE operation to remove the address.

Execute a JOIN query to fetch the address using the customer's ID.

Use the UPDATE operation to find the address.

Create a new customer entry with the same ID.

Q416. Alex wants to update the details of a specific user's account, which is linked to a unique profile in a Spring Boot application. What approach should he take to update the user's profile?

Ignore the profile since updates are not allowed for One to One relationships.

Load the user entity, modify the profile attributes, and save the user entity.

Delete the existing user profile and create a new one.

Use a batch operation to update all user profiles at once.

Q417. A software developer is implementing a feature in a Spring Boot application that requires a One-to-One association between the `Employee` and `Address` entities. To ensure that each Employee has exactly one Address, which configuration should the developer choose for the `Address` entity?

Annotate the Address entity with @OneToOne and use @JoinColumn(name = "employee_id") to create the foreign key.

Use @OneToMany on the Employee entity with @OneToOne on the Address entity.

Annotate Employee with @Embedded to include Address attributes directly.

Implement the Address as a separate entity without any annotations, only using a reference in the Employee entity.

Q418. A developer is working on an application that requires a bidirectional One-to-One relationship between a `Customer` and a `Wallet` entity. The `Wallet` entity should reference back to the `Customer`, while ensuring that both sides of the relationship are correctly maintained. How should the developer configure the `Customer` entity?

Use @OneToOne(mappedBy = "customer") on the Customer entity to define the ownership in the Wallet.

Confirm both Customer and Wallet have @OneToOne annotations without defining mapped ownership.

Implement the relationship using @ManyToOne in the Wallet entity, pointing back to Customer.

Use @OneToOne on both entities without using any additional attributes.

Q419. You are tasked with creating a RESTful API service with Spring Boot that handles user accounts. When a new User is registered, a Profile needs to be created simultaneously. What would be the best approach to ensure both entities are saved correctly while maintaining their relationship?

Use @Transactional on the service method to save the User and Profile within the same transaction.

Save the Profile first, then save the User without ensuring their relationship.

Only save the User entity and rely on a separate service to handle Profile creation later.

Call the save method for both User and Profile independently, without managing transaction boundaries.

Q420. Jessica is designing a library management system where each author can write multiple books. How should she establish a One to Many relationship in her Spring Boot JPA application?

The Author entity should have a @OneToOne annotation, while the Book entity should have a @ManyToOne annotation.

The Author entity should have a @OneToMany annotation, while the Book entity should be marked with @ManyToOne.

Both the Author and Book entities should reference each other with @ManyToMany annotations.

The Book entity should exclusively define the One to Many relationship using @OneToMany annotation.

Q421. Alice is working on a project that involves creating a web application for managing school classes. She needs to implement CRUD operations to manage classes and students, where each class can have multiple students. What is the correct sequence of actions to add a new student to an existing class?

Create the student object, save it independently, and then associate it with the class.

Update the class object with the new student in the list and then save the class entity.

Add the student to a separate database and link it to the class later.

Only save the student entity; the class will automatically link to it in the database.

Q422. Tom is implementing a database to store customer information and their respective orders. He knows that each customer can have multiple orders. Which attribute in the Order entity should Tom use to establish which customer each order belongs to?

A field indicating the total price of the order.

A field annotated with @ManyToOne referencing the Customer entity.

A field containing the order date.

A list containing all the orders in the Customer entity.

Q423. Maria is tasked with designing a bookstore application using Spring Boot JPA. She needs to establish a relationship where a single author can have multiple books. How should she define this One to Many association in her entity classes?

Use @ManyToOne in the Author entity to reference a list of books.

Implement @OneToMany in the Author entity with a List while ensuring @ManyToOne in the Book entity.

Define both entities with @OneToMany annotations.

Create a separate entity for the relationship without any annotations.

Q424. Sarah is implementing a task management system where a project can have multiple tasks. She extends the JpaRepository to manage CRUD operations for both `Project` and `Task` entities. Which statement is true regarding how she should implement the repository layer?

She does not need to implement a repository for the Task entity since it is related to Project.

Both ProjectRepository and TaskRepository should be extended from JpaRepository and JpaRepository respectively.

CRUD operations should be implemented using manual SQL queries without extending JpaRepository.

She only needs one repository for both entities to handle the CRUD operations.

Q425. Alex is integrating a feature in his application's user management system that maintains each user's posts. The posts must be retrieved based on the associated user information. After defining a `User` entity with a list of `Post` entities, he used `@OneToMany(mappedBy = "user")` in the `User` class. What does this imply about the relationship?

The User entity is responsible for managing the relationship.

The Post entity will manage the relationship and control database operations.

The relationship cannot be cascaded to the Post entity.

A join table must be used for this relationship.

Q426. Tim is working on an e-commerce application where each order can have multiple order items. He decides to use cascading to manage the lifecycle of child entities related to the parent. Which annotation configuration will effectively propagate persistence operations from the `Order` entity to the `OrderItem` entities?

Use `@OneToMany(mappedBy = "order", cascade = CascadeType.ALL)` in the Order class.

Only the OrderItem should have the cascade configuration.

Parent entities cannot have cascade settings with One to Many relationships.

Only persistence operations should be cascaded; retrieving data does not require cascades.

Q427. Jane is developing a blog application where multiple blog posts can be attributed to a single author. Which situation exemplifies a Many to One relationship in this context?

Multiple authors can contribute to a single blog post.

Each blog post can have multiple tags connected to it.

Several blog posts belong to one specific author.

A single author can write multiple comments on different posts.

Q428. Tom is designing a database for a school system where multiple students can enroll in a single class. Which description best fits this scenario regarding Many to One association?

Each student can enroll in multiple classes.

Each class can have multiple teachers.

Many students can be associated with one class.

Each teacher can teach multiple classes.

Q429. Alex is developing a web application that requires different users to have different levels of access. Which role-based access control (RBAC) statement is correct?

Users with admin roles can access all resources.

Users with user roles can access admin functionalities.

All users can view sensitive data without restrictions.

Users with guest roles can access restricted areas.

Q430. John wants to secure his Spring application by defining security rules efficiently. He discovers a specific annotation that helps in setting up Spring Security configurations within his Java classes. Which annotation should he use?

@EnableConfiguration

@EnableSecurity

@EnableWebSecurity

@SecurityConfig

Q431. A developer explains to a team that proving a user's identity occurs before determining what resources they can access. Which two concepts are they describing?

- Authentication and authorization
- Authorization and encryption
- Authentication and encryption
- User roles and permissions

Q432. Emma is tasked with implementing user authentication for her Spring application. She learns that one common method encodes credentials to avoid sending them in plain text. Which authentication mechanism is she most likely learning about?

- Token-based Authentication
- OAuth
- Basic Authentication
- Digest Authentication

Q433. A healthcare application is being developed where only authorized medical personnel should access patient records. The team decides to implement Role-Based Access Control (RBAC) and assigns roles accordingly. What would be the best approach to implement RBAC for this scenario?

- Assign all users the same permissions to simplify management.
- Implement roles such as 'Doctor' who can access full medical records and 'Nurse' who can only view basic information.
- Revoking all permissions for users until they request access.
- Allow users to self-assign roles based on their job titles.

Q434. An online marketplace is concerned about fraudulent transactions. They are looking into securing user sessions to ensure that unauthorized access is minimized. What is the most effective method for managing user sessions?

- Allowing sessions to last indefinitely to avoid user inconvenience.
- Implementing periodic re-validation of sessions, requiring users to log in again after a set timeout.
- Storing session state entirely on the client-side for faster access.
- Having users re-authenticate only after a month of inactivity.

Q435. A web application allows different user roles to access various resources. Developers are tasked with ensuring that unauthorized users cannot access sensitive sections. What would be the best way to enforce this through URL-based authorization?

- Allow unrestricted access for all users to simplify development.
- Define strict URL-based access rules in the security configuration to limit access based on user roles.
- Use a generic homepage redirection for unauthorized users as the only security measure.
- Disable all security checks to enhance application performance.

Q436. Alex is developing a Spring Boot application that requires user authentication through tokens. He needs to ensure that once a user logs in, they do not have to authenticate again for a specific period. Which of the following best describes the role of a refresh token in this scenario?

- Refresh tokens are used to log the user out of the application.
- Refresh tokens allow the user to maintain their session without re-entering their credentials.
- Refresh tokens are temporary tokens that can only be used once.
- Refresh tokens are used to directly authenticate users without an initial login.

Q437. A developer is working on an e-commerce application that utilizes token-based authentication. After a user successfully logs in, the application generates an access token and a refresh token. What is the primary purpose of the refresh token in this scenario?

- To allow users to authenticate without re-entering their credentials once the access token expires.
- To store sensitive user data such as passwords for quick access.
- To verify the authenticity of the access token without additional processing.
- To provide a backup if the access token is lost or compromised.

Q438. Sarah has implemented JWT (JSON Web Tokens) for user authentication in her Spring Boot application. She notices that there are multiple benefits to using JWT, especially for stateless authentication. Which of the following describes a major advantage of this approach in her application?

- JWT tokens are stored on the client-side, making server storage unnecessary.
- JWT tokens are easy to revoke at any time without affecting active sessions.

All data within a JWT token is automatically encrypted for security.

The application has to maintain a session store for user authentication and authorization.

Q439. Alice is developing a web application that requires user authentication. She decides to implement JWTs for handling user sessions. During the JWT generation process, she specifically includes the user's role information in the Payload. A week later, she encounters a scenario where a user claims their access has changed but the JWT they are using still includes the previous role. What could be the issue with her current implementation?

The JWT should not include role information in the Payload.

The JWT does not refresh automatically, which means changes in user roles won't be reflected until a new token is issued.

The Header must specify the user role to update it successfully.

The server can invalidate the JWT whenever there's a role change, making it stateful.

Q440. A team is deploying a critical update on their project, but one of the developers realizes they made a mistake that affected the code. They executed the command `git push origin main` after committing their changes. What should they do to rectify the mistake while minimizing disruptions?

Use `git revert` to create a new commit that undoes the changes made by the mistaken commit.

Force push the previous commit using `git push --force`.

Delete the mistaken commit from the main branch and re-push the master.

Contact the project manager to redo the entire deployment process.

Q441. Alex is working on a new feature in a project that requires extensive changes. He decides to use a branch so that he can make changes without affecting the main codebase. After some time, he completes the feature but realizes that two other developers have made changes to the main branch that could lead to merge conflicts. What should Alex consider before merging his branch back into the main branch?

Ensure his branch is up to date with the main branch and resolve any conflicts locally.

Only merge if the changes in the main branch are minor and do not affect his code.

Ignore the changes in the main branch as they are not related to his feature.

Ask the other developers to revert their changes before merging.

Q442.

Maria is teaching her new intern how to use Git. She explains that snapshots are taken at different points in time during development. The intern asks her about the role of commits. What is Maria's best explanation?

- Commits create snapshots of the entire project and store them in a designated folder.
- Commits represent individual changes and allow for the tracking of modifications over time.
- Commits only record the changes made in a single file at once.
- Commits are used solely for backing up the project before significant updates.

Q443. John has made some changes to the files in his local Git repository and is ready to save those changes. Which command should he use to preserve his changes along with a descriptive message?

- git push
- git stage
- git commit
- git pull

Q444. Lisa has made a number of commits in her local repository and wants to share her work with her team by uploading the changes to the remote repository. What command will she use to achieve this?

- git merge
- git stage
- git push
- git clone

Q445. Emma has just received a link to a remote Git repository containing a project she needs to work on. What command should she use to create a local copy of that repository on her machine?

- git pull
- git clone
- git fetch
- git init

Q446. Jessica is working on a new feature for her web application and wants to create a separate branch to manage her changes. What is the correct way for her to create a new branch in GitHub using the web interface?

Click on the "Code" tab, then type the new branch name in the "Branch: main" dropdown and click "Create branch".

Navigate to the settings, select "Branches", and create a new branch from there.

Open a terminal, use `git branch new-feature` and push the branch.

Send an email to the repository owner requesting a new branch creation.

Answer Key & Solution

Section 1 - LTM_JAVA_MCQ_PRACTICE_FOR_TRAINER

Q1 It injects a reference to the EntityManager into a Spring bean.

Solution

No Solution

Q2 It handles exceptions thrown by controller methods

Solution

No Solution

Q3 DTOs serve as lightweight data containers for transferring data between different layers of the application, promoting separation of concerns and decoupling.

Solution

No Solution

Q4 Proxy or load balancer issue

Solution

No Solution

Q5 Hibernate may generate an incorrect database schema or create duplicate relationship mappings

Solution

No Solution

Q6 It determines the propagation behavior of operations (e.g., save, delete) from parent to child entities.

Solution

No Solution

Q7 @OneToMany(mappedBy = "order", cascade = CascadeType.ALL) in Order and @ManyToOne in OrderItem.

Solution

No Solution

Q8 It defines a transactional boundary for a method or class

Solution

	No Solution
Q9	To specify that the child entity owns the relationship. Solution
	No Solution
Q10	The name of the relationship field in the owning side entity that manages the foreign key Solution
	No Solution
Q11	JWT tokens are stateless, reducing the need to store session information on the server. Solution
	No Solution
Q12	Java Native Interface (JNI) Solution
	No Solution
Q13	When the application involves managing large volumes of data with multiple interrelated entities and relationships. Solution
	No Solution
Q14	It indicates that the association is mandatory and cannot be null. Solution
	No Solution
Q15	A snapshot of the project's current state at a specific point in time Solution
	No Solution
Q16	Implement a custom exception handler using <code>@ControllerAdvice</code> to intercept <code>ProductCreationException</code> and return a structured error response with detailed error codes and messages, adhering to RESTful API best practices. Solution

	No Solution
Q17	Extend the Exception class
	Solution
	No Solution
Q18	It selects a specific bean when multiple beans of the same type are available
	Solution
	No Solution
Q19	404
	Solution
	No Solution
Q20	Unit of Work pattern
	Solution
	No Solution
Q21	It defines the foreign key column in the database table of the owning entity.
	Solution
	No Solution
Q22	To eliminate redundancy and minimize data duplication.
	Solution
	No Solution
Q23	It denotes the process of negotiating the content type and encoding between the client and server to determine the most suitable representation of the resource.
	Solution
	No Solution
Q24	init() initializes the servlet, service() handles incoming requests, and destroy() releases resources.
	Solution

	No Solution
Q25	It ensures that the primary key of the owning entity is automatically set as the primary key of the associated entity. Solution
	No Solution
Q26	Specify a custom message using the message attribute in the validation annotation. Solution
	No Solution
Q27	It specifies that duplicate entities should be excluded from the query result. Solution
	No Solution
Q28	It bootstraps the Spring Boot application and launches the application context. Solution
	No Solution
Q29	@Transient Solution
	No Solution
Q30	By configuring a centralized error handler using @ControllerAdvice and @ExceptionHandler to intercept and process exceptions globally. Solution
	No Solution
Q31	It leads to repetitive code and makes centralized error handling difficult Solution
	No Solution
Q32	@Table Solution
	No Solution

Q33	@RestController combines the behavior of @Controller and @ResponseBody, automatically converting method return values to JSON or XML responses, making it suitable for RESTful APIs. Solution No Solution
Q34	To intercept and modify HTTP requests and responses before they reach the servlet or after they leave. Solution No Solution
Q35	It allows stateless communication between the client and server Solution No Solution
Q36	UsernamePasswordAuthenticationFilter Solution No Solution
Q37	It filters the grouped results based on a specified condition. Solution No Solution
Q38	A named query parameter that will be dynamically replaced with a department name at runtime. Solution No Solution
Q39	The exception is caught, and the method returns HTTP 500 Internal Server Error Solution No Solution
Q40	Handling HTTP request mapping, request parameter parsing, and generating appropriate HTTP responses. Solution No Solution
Q41	Tomcat is a servlet container that executes Java servlets and manages JavaServer Pages (JSP).

	Solution
	No Solution
Q42	Authentication verifies user credentials, while authorization determines what resources a user can access.
	Solution
	No Solution
Q43	It denotes that the server successfully processed the request but intentionally returned no content in the response body.
	Solution
	No Solution
Q44	To update an existing resource with partial modifications.
	Solution
	No Solution
Q45	To validate user credentials and authenticate users.
	Solution
	No Solution
Q46	It indicates the version of the entity for optimistic locking.
	Solution
	No Solution
Q47	Criteria API
	Solution
	No Solution
Q48	Check server logs for specific error messages.
	Solution
	No Solution
Q49	To create a join table for a many-to-many relationship
	Solution

	No Solution
Q50	They provide metadata for dependency injection Solution
	No Solution
Q51	It may lead to performance degradation and increased network bandwidth consumption due to the size of the response payload, especially for large datasets. Solution
	No Solution
Q52	Utilize built-in Spring Data JPA features for pagination. Solution
	No Solution
Q53	@ManyToOne(fetch = FetchType.EAGER) @JoinColumn(name = "country_id") private Country country; Solution
	No Solution
Q54	Spring Transaction Management using @Transactional Solution
	No Solution
Q55	The request has been accepted for processing, but the processing has not yet been completed Solution
	No Solution
Q56	It indicates a collection of simple values or embeddable objects. Solution
	No Solution
Q57	It provides endpoints for exposing application metrics and status information.

Q58	Solution
	No Solution
	@JoinColumn
Q59	Solution
	No Solution
	Identifying the root cause by examining the stack trace
Q60	Solution
	No Solution
	Objects
Q61	Solution
	No Solution
	Introducing redundancy and duplication in a database schema to optimize data retrieval and manipulation.
Q62	Solution
	No Solution
	They provide a centralized location to define exception handling logic for multiple controllers.
Q63	Solution
	No Solution
	To store claims (such as user information and token metadata)
Q64	Solution
	No Solution
	To specify the order in which filters should be applied.
Q65	Solution
	No Solution
	Implementing business logic and interacting with repositories for data operations.
	Solution

	No Solution
Q66	It is a feature that allows Spring to automatically detect and inject dependencies into beans. Solution No Solution
Q67	By using @OneToOne annotation in the User entity and @OneToOne(mappedBy = "user") in the Profile entity. Solution No Solution
Q68	Third Normal Form (3NF) Solution No Solution
Q69	Use @OneToMany(mappedBy = "order", cascade = CascadeType.ALL) in Order. Solution No Solution
Q70	To specify that the field should be excluded from JSON serialization and deserialization processes, preventing sensitive data from being exposed in API responses. Solution No Solution
Q71	SecurityContext Solution No Solution
Q72	401 Unauthorized Solution No Solution
Q73	@Query(value = "SELECT * FROM table_name", nativeQuery = true) Solution

	No Solution
Q74	It denotes the logical partitioning of application functionality into distinct layers such as presentation, business logic, and data access, promoting separation of concerns and maintainability. Solution
	No Solution
Q75	TRACE Solution
	No Solution
Q76	The IoC container Solution
	No Solution
Q77	Guaranteed better runtime performance in all scenarios Solution
	No Solution
Q78	@ControllerAdvice with @ExceptionHandler Solution
	No Solution
Q79	Aggregating query results based on common property values. Solution
	No Solution
Q80	It indicates whether the association should be eagerly or lazily fetched. Solution
	No Solution
Q81	It indicates that the User object should be validated against predefined constraints specified in the domain model before being processed. Solution

	No Solution
Q82	To represent the details of a user, including username, password, and authorities. Solution No Solution
Q83	Interacting with the database to perform CRUD operations and managing data persistence. Solution No Solution
Q84	It represents a connection to the database and manages transactions. Solution No Solution
Q85	It specifies a bean for database repository operations. Solution No Solution
Q86	To define the interface, operations, and communication details of a web service Solution No Solution
Q87	@EnableJpaRepositories Solution No Solution
Q88	By using named parameters (e.g., :param1, :param2) Solution No Solution
Q89	The child entity is the owning side because it contains the foreign key. Solution No Solution

Q90	By defining foreign key relationships between database tables.
	Solution
	No Solution
Q91	@Entity
	Solution
	No Solution
Q92	To provide health checks and monitoring endpoints.
	Solution
	No Solution
Q93	ERROR
	Solution
	No Solution
Q94	Managing the lifecycle, configuration, and dependencies of Spring beans
	Solution
	No Solution
Q95	Web services facilitate communication between different applications over a network using standardized protocols.
	Solution
	No Solution
Q96	Spring MVC HTTP Message Converters
	Solution
	No Solution
Q97	It allows different implementations of an interface to be injected and used through a common interface at runtime
	Solution
	No Solution
Q98	By annotating a method in a @ControllerAdvice class with @ExceptionHandler for the specific exception type.

Solution

No Solution

Q99

To simplify the configuration and deployment of Spring applications.

Solution

No Solution

Q100

java.time.LocalDate

Solution

No Solution

Q101

Use nested @Valid annotations in the request body and define appropriate validation constraints.

Solution

No Solution

Q102

The Spring context will fail to start, and the bean will not be initialized

Solution

No Solution

Q103

@ConditionalOnProperty

Solution

No Solution

Q104

ServletContextAttributeListener

Solution

No Solution

Q105

The value of {id} in the URL is injected into the method parameter

Solution

No Solution

Q106

Cache-Control

Solution

	No Solution
Q107	@Transactional(readOnly = true) and @Transactional(readOnly = false) Solution No Solution
Q108	SELECT * FROM table1 JOIN table2 ON table1.id = table2.id; Solution No Solution
Q109	Use URL path versioning (e.g., /api/v1/resource) and maintain separate controllers for each version. Solution No Solution
Q110	Use WebSockets or Server-Sent Events (SSE) to push updates to clients. Solution No Solution
Q111	By configuring HttpMessageConverter beans Solution No Solution
Q112	ROLE_ Solution No Solution
Q113	To specify the base packages for scanning Spring beans Solution No Solution
Q114	It uses @RequestBody to bind JSON or XML data to the complex object Solution No Solution

Q115	HS256
	Solution
	No Solution
Q116	WebSecurityConfigurerAdapter
	Solution
	No Solution
Q117	Use @RequestHeader to read the Accept-Language header and adjust the response.
	Solution
	No Solution
Q118	Specifies the MIME type that a method returns
	Solution
	No Solution
Q119	Upgrade
	Solution
	No Solution
Q120	Students and their courses, including those with no enrollments or courses.
	Solution
	No Solution
Q121	By exchanging messages in a predefined format over HTTP.
	Solution
	No Solution
Q122	A Java program that extends the functionality of a web server by handling client requests.
	Solution
	No Solution
Q123	To host and execute Java servlets and JSPs.

	Solution
	No Solution
Q124	JSON is more efficient and lightweight than XML.
	Solution
	No Solution
Q125	XML is a markup language while JSON is a lightweight data-interchange format.
	Solution
	No Solution
Q126	{ "name": "John", "age": 30 }
	Solution
	No Solution
Q127	Increased dependence on proprietary protocols.
	Solution
	No Solution
Q128	By converting objects into SQL statements and executing them against the database.
	Solution
	No Solution
Q129	Hibernate is a specific ORM implementation, while JPA is a standard specification.
	Solution
	No Solution
Q130	To automatically generate values for the primary key.
	Solution
	No Solution
Q131	By using foreign keys to link related objects in the database.
	Solution

	No Solution
Q132	To map an object class to a database table. Solution
	No Solution
Q133	JPQL Solution
	No Solution
Q134	Using the EntityManager.remove() Solution
	No Solution
Q135	To define the primary key of an entity. Solution
	No Solution
Q136	All of the above. Solution
	No Solution
Q137	Using the EntityManager.merge() Solution
	No Solution
Q138	To define a many-to-one relationship between entities. Solution
	No Solution
Q139	To reduce data redundancy and improve data integrity. Solution
	No Solution

Q140	Simplified data access for users.
	Solution
	No Solution
Q141	Increased complexity of database design.
	Solution
	No Solution
Q142	First Normal Form (1NF)
	Solution
	No Solution
Q143	Adding redundant data to improve performance.
	Solution
	No Solution
Q144	Data is organized hierarchically in a tree-like structure.
	Solution
	No Solution
Q145	A foreign key in one table must match a primary key in another table.
	Solution
	No Solution
Q146	All non-key attributes are dependent on the primary key.
	Solution
	No Solution
Q147	That relationships between tables are correctly enforced.
	Solution
	No Solution
Q148	All of the above.

Solution

No Solution

Q149

Tracking changes to files and allowing for collaboration on projects.

Solution

No Solution

Q150

Preventing accidental changes to files by locking them for editing.

Solution

No Solution

Q151

To save a snapshot of the current state of the project's files.

Solution

No Solution

Q152

Git is a command-line tool, while GitHub is a web-based platform.

Solution

No Solution

Q153

Fetch mode

Solution

No Solution

Q154

Named queries are faster as they are precompiled

Solution

No Solution

Q155

The service() method for handling all types of requests.

Solution

No Solution

Q156

By using the Accept header in the HTTP request.

No Solution

Q157

Child entities are not affected by parent operations

Solution

No Solution

Q158

Constructor injection for both notifications

Solution

No Solution

Q159

SELECT d.name, SUM(e.salary) FROM Department d JOIN d.employees e GROUP BY d.name

Solution

No Solution

Q160

A condition where one attribute uniquely determines another

Solution

No Solution

Q161

All CommandLineRunner beans are executed sequentially based on their @Order annotation

Solution

No Solution

Q162

200 OK with the item details

Solution

No Solution

Q163

findUserByEmail(String email)

Solution

No Solution

Q164

@OneToMany

Solution

No Solution

Q165 Use http.authorizeRequests().anyRequest().authenticated() in the security configuration

Solution

No Solution

Q166 Implement the Specification interface for building queries.

Solution

No Solution

Q167 Merge conflicts occur when changes in different branches affect the same line of code, requiring manual resolution.

Solution

No Solution

Q168 Implement a refresh token mechanism to obtain a new access token without re-authentication.

Solution

No Solution

Q169 Implement a role-based access control mechanism by validating roles decoded from the JWT at request time.

Solution

No Solution

Q170 Check the integrity of claims against trusted server-side data before processing requests.

Solution

No Solution

Q171 Use a CSRF token and include it in all state-changing requests

Solution

No Solution

Q172 Joined inheritance mapping to separate attributes into relevant tables while sharing common attributes.

Solution

No Solution

Q173 @Component

	Solution
	No Solution
Q174	Use the @RestController along with @RequestMapping for each endpoint.
	Solution
	No Solution
Q175	The @Entity annotation marks a class as a persistent entity for database mapping.
	Solution
	No Solution
Q176	Denormalization, including redundant data, to improve read performance for analytical queries
	Solution
	No Solution
Q177	It is a DAO that interacts with the database
	Solution
	No Solution
Q178	repository.save(category)
	Solution
	No Solution
Q179	5
	Solution
	No Solution
Q180	@NamedQuery
	Solution
	No Solution
Q181	findByLastName

	Solution
	No Solution
Q182	Create a branch named "feature-update" and switch to it for making all changes.
	Solution
	No Solution
Q183	It allows for easy implementation of CRUD operations without boilerplate code.
	Solution
	No Solution
Q184	Branches
	Solution
	No Solution
Q185	@PreAuthorize
	Solution
	No Solution
Q186	B only
	Solution
	No Solution
Q187	XML and JSON
	Solution
	No Solution
Q188	Use the application.properties or application.yml file with profile-specific configurations.
	Solution
	No Solution
Q189	It enables tracking changes to files over time.
	Solution

	No Solution
Q190	Normalize the database by moving repeating customer data into its own table
	Solution
	No Solution
Q191	LAZY
	Solution
	No Solution
Q192	Use EAGER fetching for product categories while using LAZY fetching for associated products to reduce the initial fetch time.
	Solution
	No Solution
Q193	Use @ManyToMany between students and courses, and @ManyToOne from Course to Instructor
	Solution
	No Solution
Q194	Create a configuration class and use @EnableWebSecurity along with configure() methods.
	Solution
	No Solution
Q195	It is placed on the inverse (non-owning) side to tell JPA that the foreign key is managed by the field named on the other entity
	Solution
	No Solution
Q196	git commit
	Solution
	No Solution
Q197	@Configuration
	Solution
	No Solution

Q198	Data normalization
	Solution
	No Solution
Q199	Create a new branch for the feature and work on it independently.
	Solution
	No Solution
Q200	To represent tables in a relational database.
	Solution
	No Solution
Q201	Pull the main branch into the feature branch and resolve any merge conflicts that arise, ensuring integration is complete before merging into the main branch.
	Solution
	No Solution
Q202	Open a pull request against the main branch to initiate the code review process.
	Solution
	No Solution
Q203	@Query
	Solution
	No Solution
Q204	Create a Specification that combines different criteria using predicates for a flexible query construction.
	Solution
	No Solution
Q205	A list of books, including their author IDs but excluding publisher details.
	Solution
	No Solution
Q206	JPQL is specifically designed to work with Java objects instead of database tables.

	Solution
	No Solution
Q207	@Embeddable
	Solution
	No Solution
Q208	Creating a branch
	Solution
	No Solution
Q209	A only
	Solution
	No Solution
Q210	@Query("SELECT p FROM Product p WHERE p.available = true ORDER BY p.price ASC") List<Product> findAvailableProductsSortedByPrice();
	Solution
	No Solution
Q211	Roles define access levels for users.
	Solution
	No Solution
Q212	Implement Git Flow strategy, creating dedicated branches for feature, develop, and master.
	Solution
	No Solution
Q213	findByStockEquals(0)
	Solution
	No Solution
Q214	Issue

	Solution
	No Solution
Q215	Spring Boot Starter
	Solution
	No Solution
Q216	A repository interface simplifies data access by providing CRUD operations automatically.
	Solution
	No Solution
Q217	B only
	Solution
	No Solution
Q218	Using the @Param annotation with a method parameter.
	Solution
	No Solution
Q219	Java Persistence Query Language
	Solution
	No Solution
Q220	It provides an easy way to implement data access layers using repositories.
	Solution
	No Solution
Q221	Eager fetching retrieves data immediately.
	Solution
	No Solution
Q222	A only
	Solution

No Solution

Q223
B only

Solution

No Solution

Q224
It reduces data redundancy and improves data integrity

Solution

No Solution

Q225
SELECT

Solution

No Solution

Q226
SELECT e FROM Employee e WHERE e.salary > :salary

Solution

No Solution

Q227
@RestController

Solution

No Solution

Q228
REST uses standard HTTP methods, while SOAP relies on XML messaging and is protocol-based.

Solution

No Solution

Q229
Extend JpaRepository and create repository interfaces for data access.

Solution

No Solution

Q230
@JoinColumn

Solution

No Solution

Q231	Create a new branch	Solution No Solution
Q232	To define the name of the database table that the entity maps to.	Solution No Solution
Q233	C	Solution No Solution
Q234	@Entity	Solution No Solution
Q235	@OneToMany	Solution No Solution
Q236	Save changes locally and push to remote	Solution No Solution
Q237	Testing each feature branch independently, then merging them into the release branch while ensuring everything integrates smoothly.	Solution No Solution
Q238	Using InMemoryUserDetailsManager for user storage	Solution No Solution
Q239	The @Id annotation is missing on the id field, so it cannot be recognized as the primary key	

Solution

No Solution

Q240

Service Provider, Service Requester, and Service Registry

Solution

No Solution

Q241

First Normal Form (1NF)

Solution

No Solution

Q242

B only

Solution

No Solution

Q243

@ManyToOne

Solution

No Solution

Q244

@SpringBootApplication

Solution

No Solution

Q245

@Transactional

Solution

No Solution

Q246

A primary key

Solution

No Solution

Q247

application.properties

No Solution

Q248The total number of orders matching the given status

Solution

No Solution

Q249B only

Solution

No Solution

Q250@EnableAutoConfiguration

Solution

No Solution

Q251CSRF protection prevents unauthorized commands from being executed by users.

Solution

No Solution

Q252Foreign key

Solution

No Solution

Q253Filters process security concerns based on request and response.

Solution

No Solution

Q254A unique mapping of courses to their instructors, minimizing redundancy.

Solution

No Solution

Q255@RestController

Solution

No Solution

Q256	Only students who are enrolled in courses
	Solution
	No Solution
Q257	C only
	Solution
	No Solution
Q258	SQL (Structured Query Language)
	Solution
	No Solution
Q259	B only
	Solution
	No Solution
Q260	A only
	Solution
	No Solution
Q261	The Spring Boot Starter Data JPA simplifies dependency management.
	Solution
	No Solution
Q262	name
	Solution
	No Solution
Q263	git pull
	Solution
	No Solution
Q264	By reviewing the conflicting files, discussing the changes with the team, and selecting which changes to keep.

	Solution
	No Solution
Q265	Prints "User not found"
	Solution
	No Solution
Q266	DEFAULT
	Solution
	No Solution
Q267	Create separate tables for orders and products
	Solution
	No Solution
Q268	500
	Solution
	No Solution
Q269	One-to-Many
	Solution
	No Solution
Q270	A relational database management system (RDBMS)
	Solution
	No Solution
Q271	Spring Data JPA repositories simplify data access.
	Solution
	No Solution
Q272	Using constructor-based injection.
	Solution

	No Solution
Q273	It is used to define custom SQL queries. Solution
	No Solution
Q274	@GetMapping("/api/products") public List<Product> getAllProducts() { ... } Solution
	No Solution
Q275	Spring IoC Solution
	No Solution
Q276	Use a three-way merge to identify changes made in both branches Solution
	No Solution
Q277	It marks a class as a persistent entity. Solution
	No Solution
Q278	git merge feature Solution
	No Solution
Q279	JpaRepository Solution
	No Solution
Q280	spring-context.xml Solution
	No Solution

Q281	Java-based configuration
	Solution
	No Solution
Q282	Correct
	Solution
	No Solution
Q283	<pre>@PostMapping("/users") public User createUser(@Valid @RequestBody User user) { return userRepository.save(user); }</pre>
	Solution
	No Solution
Q284	Spring ORM
	Solution
	No Solution
Q285	@Primary
	Solution
	No Solution
Q286	@Size(min=3, max=20)
	Solution
	No Solution
Q287	git diff
	Solution
	No Solution
Q288	Git allows version control of all project files, including code.
	Solution

No Solution

Q289Object-Relational Mapping

Solution

No Solution

Q290PUT

Solution

No Solution

Q291POST is used to create new resources.

Solution

No Solution

Q292Spring MVC

Solution

No Solution

Q293@Id

Solution

No Solution

Q294@Qualifier

Solution

No Solution

Q295@Entity

Solution

No Solution

Q296/api/books

Solution

No Solution

Q297	Maven
	Solution
	No Solution
Q298	git status
	Solution
	No Solution
Q299	@PostMapping
	Solution
	No Solution
Q300	findAll()
	Solution
	No Solution
Q301	POST
	Solution
	No Solution
Q302	git checkout feature-x
	Solution
	No Solution
Q303	Map
	Solution
	No Solution
Q304	@PostMapping("/api/products") public ResponseEntity<Product> createProduct(@RequestBody Product product) { ... }
	Solution
	No Solution
Q305	@Component

	Solution
	No Solution
Q306	As a path variable
	Solution
	No Solution
Q307	A server-side program that extends the functionality of a web server
	Solution
	No Solution
Q308	By configuring Jackson2ObjectMapperBuilder bean
	Solution
	No Solution
Q309	Constructor injection is preferred over setter injection for mandatory dependencies.
	Solution
	No Solution
Q310	The code runs without errors and successfully injects the MyOtherBean bean into the MyBean bean.
	Solution
	No Solution
Q311	503
	Solution
	No Solution
Q312	Java
	Solution
	No Solution
Q313	<modules>
	Solution

No Solution

Q314

There is conflicting code between the two branches that needs resolution.

Solution

No Solution

Q315

@Size(min=1)

Solution

No Solution

Q316

@Service

Solution

No Solution

Q317

doGet()

Solution

No Solution

Q318

POST /api/books

Solution

No Solution

Q319

To name and identify objects

Solution

No Solution

Q320

To define persistence operations and manage entity lifecycle.

Solution

No Solution

Q321

To define basic CRUD (Create, Read, Update, Delete) operations for an entity.

Solution

No Solution

Q322	@NotNull
	Solution
	No Solution
Q323	To indicate a dependency that should be automatically injected
	Solution
	No Solution
Q324	<build>
	Solution
	No Solution
Q325	/api/users/123
	Solution
	No Solution
Q326	@GetMapping("/api/user/{id}") public ResponseEntity<User> getUserById(@PathVariable Long id) { ... }
	Solution
	No Solution
Q327	GET
	Solution
	No Solution
Q328	Singleton
	Solution
	No Solution
Q329	@Autowired
	Solution
	No Solution
Q330	The requested entity does not exist in the database.

	Solution
	No Solution
Q331	<form method="post">
	Solution
	No Solution
Q332	200
	Solution
	No Solution
Q333	GET
	Solution
	No Solution
Q334	To resolve bean ambiguities
	Solution
	No Solution
Q335	It handles all HTTP requests and responses.
	Solution
	No Solution
Q336	It will throw an exception of type UserNotFoundException.
	Solution
	No Solution
Q337	GET /api/users
	Solution
	No Solution
Q338	To define persistence operations and manage entity lifecycle.
	Solution

	No Solution
Q339	Static
	Solution
	No Solution
Q340	List
	Solution
	No Solution
Q341	The code runs without errors and wraps the updateSomething() method in a transaction.
	Solution
	No Solution
Q342	All of the above
	Solution
	No Solution
Q343	They eliminate the need for writing boilerplate code for common data access operations.
	Solution
	No Solution
Q344	JPA provides a standard API, allowing for vendor-independent code.
	Solution
	No Solution
Q345	Hibernate is a specific implementation of JPA.
	Solution
	No Solution
Q346	Data being inserted violates constraints set in the database schema.
	Solution
	No Solution

Q347 POST /users

Solution

No Solution

Q348 @Bean public DataSource dataSource() { ... }

Solution

No Solution

Q349 Spring Security

Solution

No Solution

Q350 The IoC container can create and manage beans with different scopes, including singleton and prototype

Solution

No Solution

Q351 POST

Solution

No Solution

Q352 Spring Core

Solution

No Solution

Q353 To simplify database access

Solution

No Solution

Q354 @Id

Solution

No Solution

Q355 BeanFactory, ApplicationContext

	Solution
	No Solution
Q356	Resource not found
	Solution
	No Solution
Q357	List<User> findByEmail(String email);
	Solution
	No Solution
Q358	By using the Pageable interface with Sort.Order(price, Sort.Direction.DESC) in the repository method.
	Solution
	No Solution
Q359	201
	Solution
	No Solution
Q360	Use @ManyToOne annotation on the Order entity and @OneToMany on the Customer entity.
	Solution
	No Solution
Q361	@Autowired
	Solution
	No Solution
Q362	It indicates that the method handles a request with a JSON payload.
	Solution
	No Solution
Q363	HTTP
	Solution

No Solution

Q364

It serves as the main interface for accessing and interacting with the Spring IoC container.

Solution

No Solution

Q365

@GetMapping

Solution

No Solution

Q366

@Transactional

Solution

No Solution

Q367

The client sends a request to the server.

Solution

No Solution

Q368

To enable interoperable machine-to-machine communication

Solution

No Solution

Q369

<bean>

Solution

No Solution

Q370

Autowiring by Type

Solution

No Solution

Q371

Constructor Injection

Solution

No Solution

Q372	Constructor Injection, ensuring the database connection is passed at the time of object creation.
	Solution
	No Solution
Q373	Dependency Injection
	Solution
	No Solution
Q374	Auto-configuration
	Solution
	No Solution
Q375	Select the "POST" method, ensure the URL points to the correct endpoint, and include the book's JSON representation in the body.
	Solution
	No Solution
Q376	GET
	Solution
	No Solution
Q377	@ResponseBody
	Solution
	No Solution
Q378	200 OK
	Solution
	No Solution
Q379	500 Internal Server Error
	Solution
	No Solution
Q380	404 Not Found

	Solution
	No Solution
Q381	The requested user ID does not exist in the database.
	Solution
	No Solution
Q382	The resource was found, but there was an issue with the database operation.
	Solution
	No Solution
Q383	A custom exception class extends the Exception class to define specific error conditions.
	Solution
	No Solution
Q384	Spring Boot provides a default error response to the client.
	Solution
	No Solution
Q385	Additional fields to store account details and the required amount to make the exception context richer.
	Solution
	No Solution
Q386	Set
	Solution
	No Solution
Q387	HashMap with ISBN as the key
	Solution
	No Solution
Q388	ORM provides automatic mapping of objects to database tables, reducing manual coding effort.
	Solution

	No Solution
Q389	ORM simplifies transaction management through built-in methods, reducing boilerplate code. Solution
	No Solution
Q390	ORM abstracts away SQL complexities, allowing developers to focus on business logic. Solution
	No Solution
Q391	@Entity Solution
	No Solution
Q392	@OneToMany Solution
	No Solution
Q393	@Id Solution
	No Solution
Q394	@Entity, @Table(name="books"), and @Id and @GeneratedValue Solution
	No Solution
Q395	@Entity Solution
	No Solution
Q396	@GeneratedValue Solution
	No Solution

Q397 @Column

Solution

No Solution

Q398 @Id

Solution

No Solution

Q399 getBooksByAuthorLastName

Solution

No Solution

Q400 @OneToMany

Solution

No Solution

Q401 LAZY

Solution

No Solution

Q402 @Transactional

Solution

No Solution

Q403 SELECT p FROM Product p

Solution

No Solution

Q404 SELECT AVG(b.price) FROM Book b

Solution

No Solution

Q405 SELECT u FROM User u WHERE u.age > 30

	Solution
	No Solution
Q406	SELECT o, c FROM Order o JOIN o.customer c
	Solution
	No Solution
Q407	SELECT s.category, SUM(s.amount) FROM Sales s GROUP BY s.category
	Solution
	No Solution
Q408	SELECT COUNT(o) FROM Order o WHERE o.customer.id = :customerId
	Solution
	No Solution
Q409	SELECT u FROM User u INNER JOIN u.orders o
	Solution
	No Solution
Q410	UPDATE User u SET u.email = :email WHERE u.id = :id
	Solution
	No Solution
Q411	Normalization helps eliminate duplicate data, ensuring efficient storage and maintenance.
	Solution
	No Solution
Q412	1NF
	Solution
	No Solution
Q413	Lucy's data is denormalized, leading to multiple entries of the same product.
	Solution

No Solution

Q414 Use the @OneToOne annotation between the User and Profile entities.

Solution

No Solution

Q415 Execute a JOIN query to fetch the address using the customer's ID.

Solution

No Solution

Q416 Load the user entity, modify the profile attributes, and save the user entity.

Solution

No Solution

Q417 Annotate the Address entity with @OneToOne and use @JoinColumn(name = "employee_id") to create the foreign key.

Solution

No Solution

Q418 Use @OneToOne(mappedBy = "customer") on the Customer entity to define the ownership in the Wallet.

Solution

No Solution

Q419 Use @Transactional on the service method to save the User and Profile within the same transaction.

Solution

No Solution

Q420 The Author entity should have a @OneToMany annotation, while the Book entity should be marked with @ManyToOne.

Solution

No Solution

Q421 Update the class object with the new student in the list and then save the class entity.

Solution

No Solution

Q422 A field annotated with @ManyToOne referencing the Customer entity.

Solution

No Solution

Q423 Implement @OneToMany in the Author entity with a List while ensuring @ManyToOne in the Book entity.

Solution

No Solution

Q424 Both ProjectRepository and TaskRepository should be extended from JpaRepository and JpaRepository respectively.

Solution

No Solution

Q425 The Post entity will manage the relationship and control database operations.

Solution

No Solution

Q426 Use @OneToMany(mappedBy = "order", cascade = CascadeType.ALL) in the Order class.

Solution

No Solution

Q427 Several blog posts belong to one specific author.

Solution

No Solution

Q428 Many students can be associated with one class.

Solution

No Solution

Q429 Users with admin roles can access all resources.

Solution

No Solution

Q430 @EnableWebSecurity

	Solution
	No Solution
Q431	Authentication and authorization
	Solution
	No Solution
Q432	Basic Authentication
	Solution
	No Solution
Q433	Implement roles such as 'Doctor' who can access full medical records and 'Nurse' who can only view basic information.
	Solution
	No Solution
Q434	Implementing periodic re-validation of sessions, requiring users to log in again after a set timeout.
	Solution
	No Solution
Q435	Define strict URL-based access rules in the security configuration to limit access based on user roles.
	Solution
	No Solution
Q436	Refresh tokens allow the user to maintain their session without re-entering their credentials.
	Solution
	No Solution
Q437	To allow users to authenticate without re-entering their credentials once the access token expires.
	Solution
	No Solution
Q438	JWT tokens are stored on the client-side, making server storage unnecessary.
	Solution

	No Solution
Q439	The JWT does not refresh automatically, which means changes in user roles won't be reflected until a new token is issued. Solution
	No Solution
Q440	Use git revert to create a new commit that undoes the changes made by the mistaken commit. Solution
	No Solution
Q441	Ensure his branch is up to date with the main branch and resolve any conflicts locally. Solution
	No Solution
Q442	Commits represent individual changes and allow for the tracking of modifications over time. Solution
	No Solution
Q443	git commit Solution
	No Solution
Q444	git push Solution
	No Solution
Q445	git clone Solution
	No Solution
Q446	Click on the "Code" tab, then type the new branch name in the "Branch: main" dropdown and click "Create branch". Solution
	No Solution